



| 300
ЛЕТ СПбГУ

Steam-ML

Студенты группы 23.Б15-пу

Абрахин Е. Д.

Бек В. А.

Доступные методы парсинга Steam данных Steam Web API

- **GetOwnedGames** — список всех игр пользователя + общее время игры
- **GetRecentlyPlayedGames** — недавно сыгранные игры за последние 2 недели
- **GetFriendList** — список друзей пользователя
- **GetPlayerSummaries** — данные профиля (ник, аватар, страна, статус)
- **GetPlayerBans** — информация о банах (VAC, игровые баны)
- **GetPlayerAchievements** — достижения игрока в конкретной игре
- **GetUserStatsForGame** — те же достижения, но без даты получения

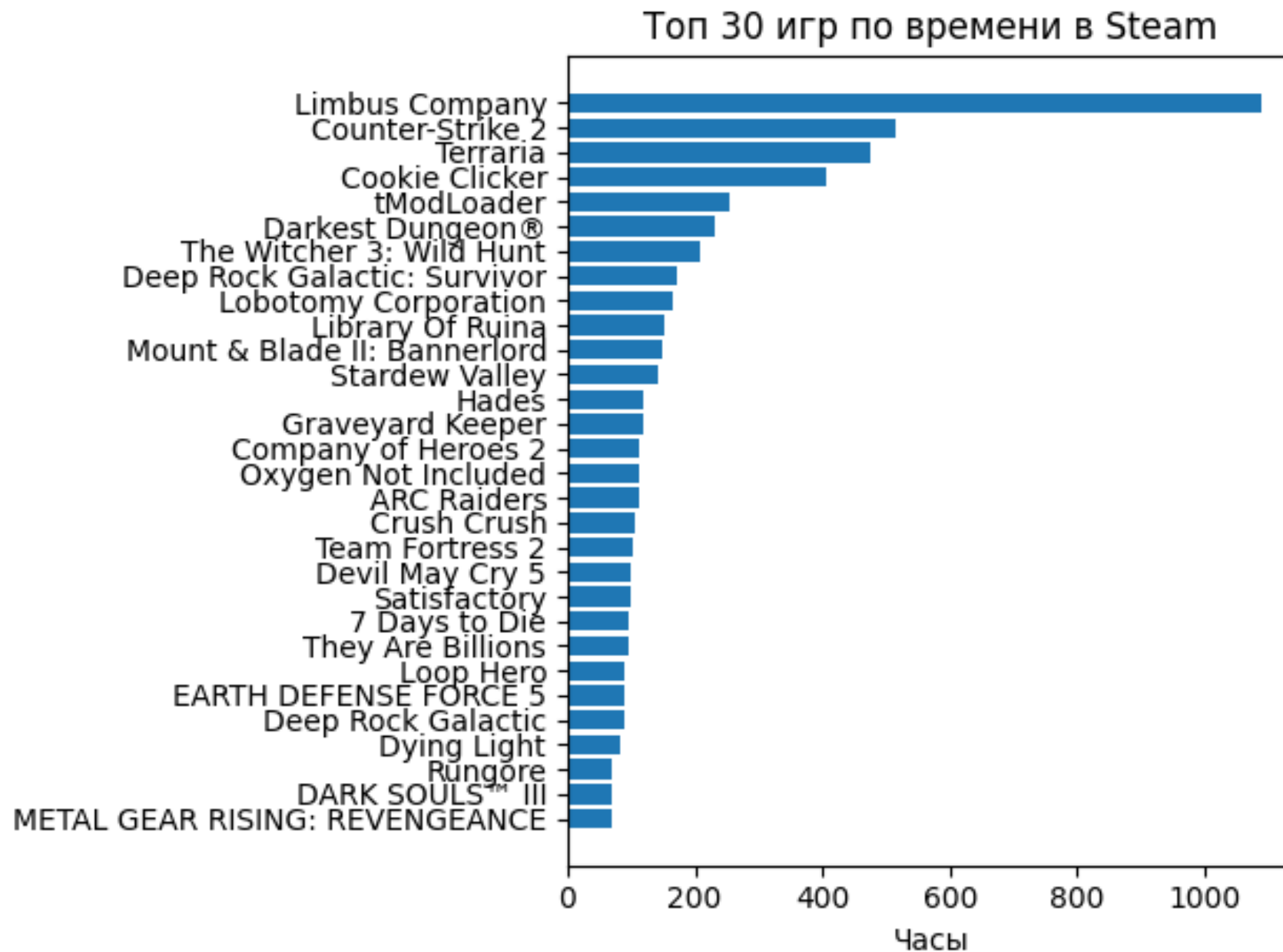
```
url = "http://api.steampowered.com/IPlayerService/GetOwnedGames/v0001/"  
params = { "key": steamApiKey, "steamid": startingSteamId,  
           "include_appinfo": True, "format": "json" }
```

Доступные методы парсинга Steam данных

Дополнительные источники

- **Steam Store API (appdetails)** — данные страницы игры (описание, цена, скриншоты, теги)
- **SteamSpy API** — аналитика игры (кол-во владельцев, теги, пики онлайн, отзывы)
- **Steam Reviews API** — статистика отзывов (положительные/отрицательные, количество)

Пример результатов запроса



Парсинг данных пользователей Steam

Модуль `steamDataCollector` собирает данные о пользователях и их игровых библиотеках через краулинг Steam.

Как работает сбор:

- Начинает с заданного пользователя и постепенно расширяет выборку через список друзей
- Для каждого найденного аккаунта запрашивает информацию о профиле и библиотеке игр
- Сбор происходит параллельно для ускорения процесса
- Поддерживает возобновление работы с предыдущей сессии и защиту от ограничений API

Что собирается:

- Данные профиля пользователя (никнейм, страна, дата регистрации)
- Список имеющихся игр и время, проведённое в каждой из них

Результат: Датасет из пользователей со статистикой их игровых предпочтений.

Парсинг данных игр Steam

После сбора пользовательских данных модуль формирует датасет по всем уникальным играм, которые встречаются в библиотеках.

Как работает сбор:

- Извлекает все уникальные идентификаторы игр из собранных пользовательских библиотек
- Для каждой игры запрашивает расширенную информацию из внешнего источника (SteamSpy)
- Сбор выполняется параллельно для повышения скорости

Что собирается:

- Основная информация об игре (название, жанр, поддерживаемые языки)
- Статистические данные (количество владельцев, количество отзывов)
- Дополнительные метрики (среднее и медианное время игры, популярные теги)

Результат: Датасет из игр со статистикой и метаданными.

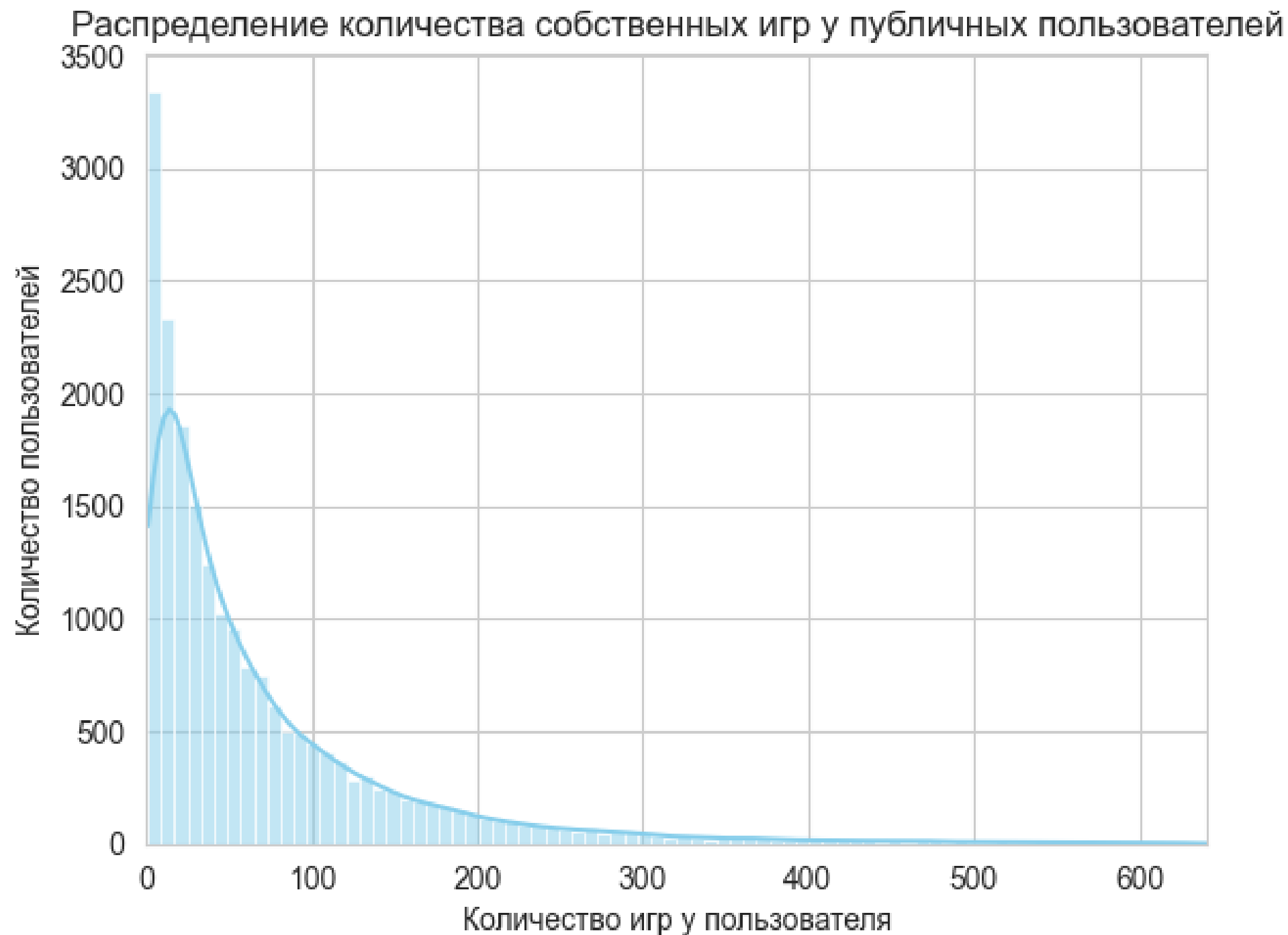
В рамках проекта было собрано два основных датасета:

Датасет пользователей

- Всего профилей: **40 000**
- Публичных профилей (доступны для анализа): **20 390** (51%)
- Среднее количество игр на пользователя: **83,9** (медиана — 40)
- Общее количество игр у всех пользователей: **1,71 млн**
- Средний год создания аккаунта: **2016**

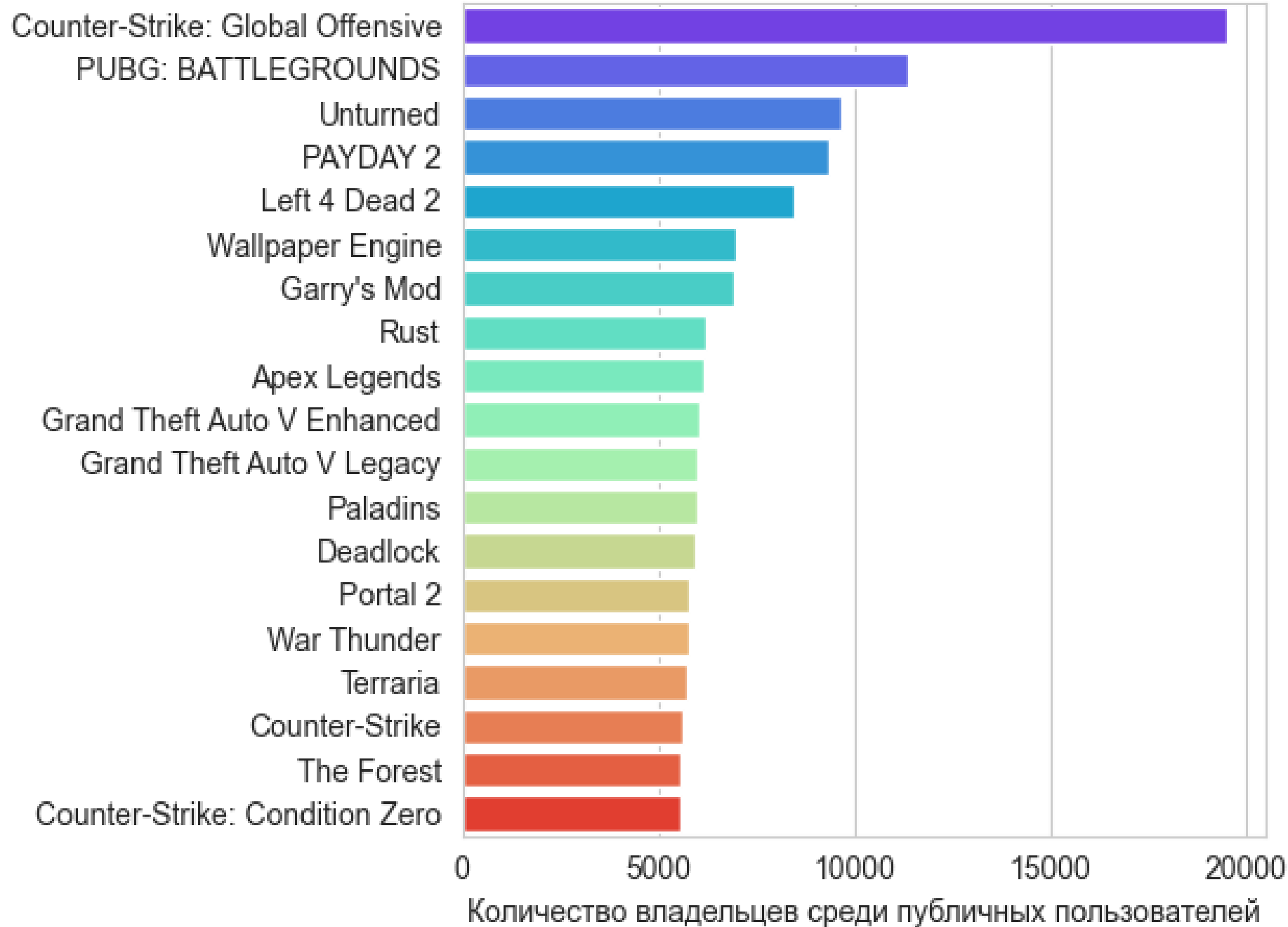
Датасет игр

- Уникальных игр: **33 233**
- Самая популярная игра: **Counter-Strike 2** (владеют **19 530** пользователей)

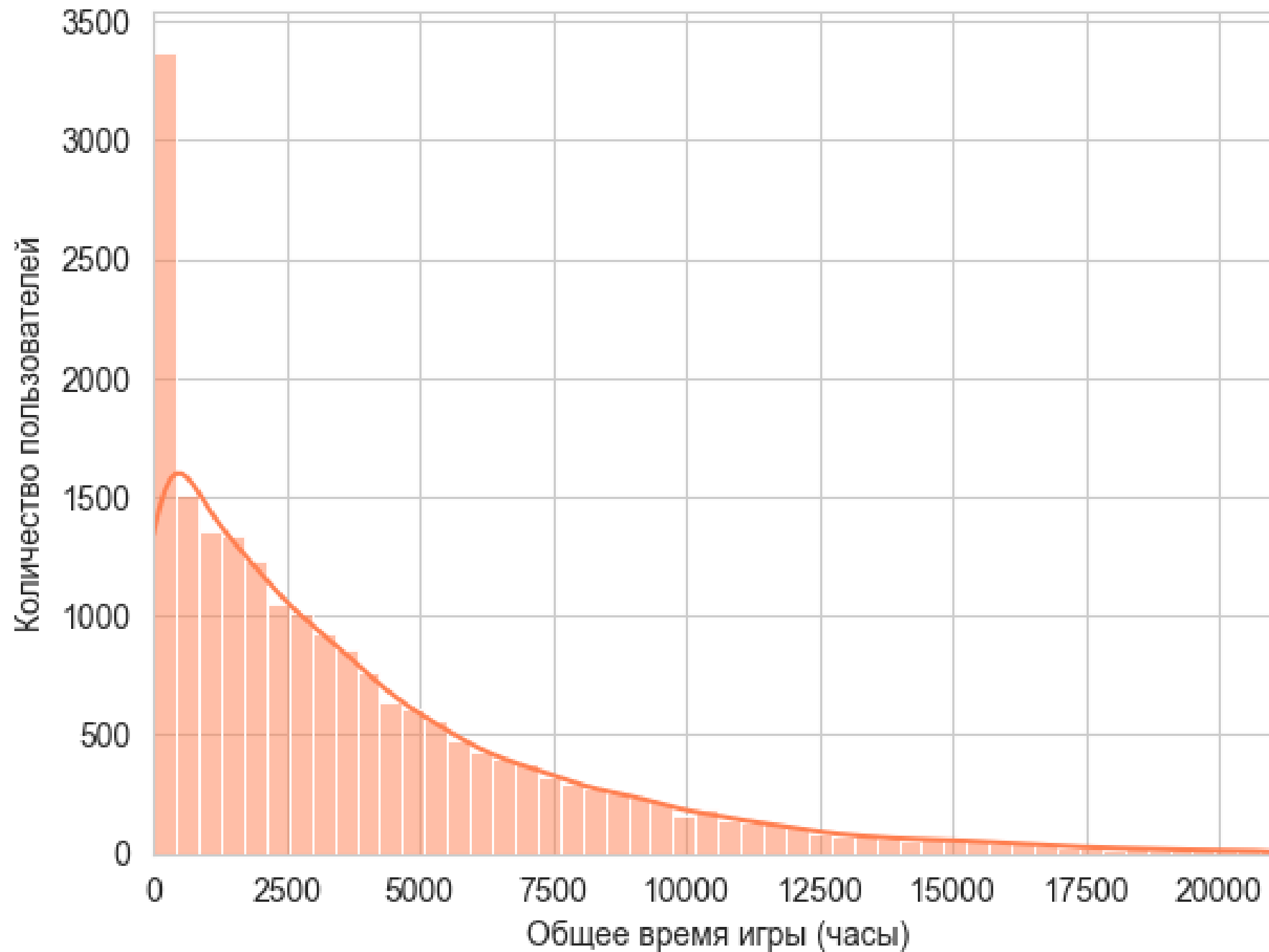


Топ-20 самых популярных игр (по количеству владельцев)

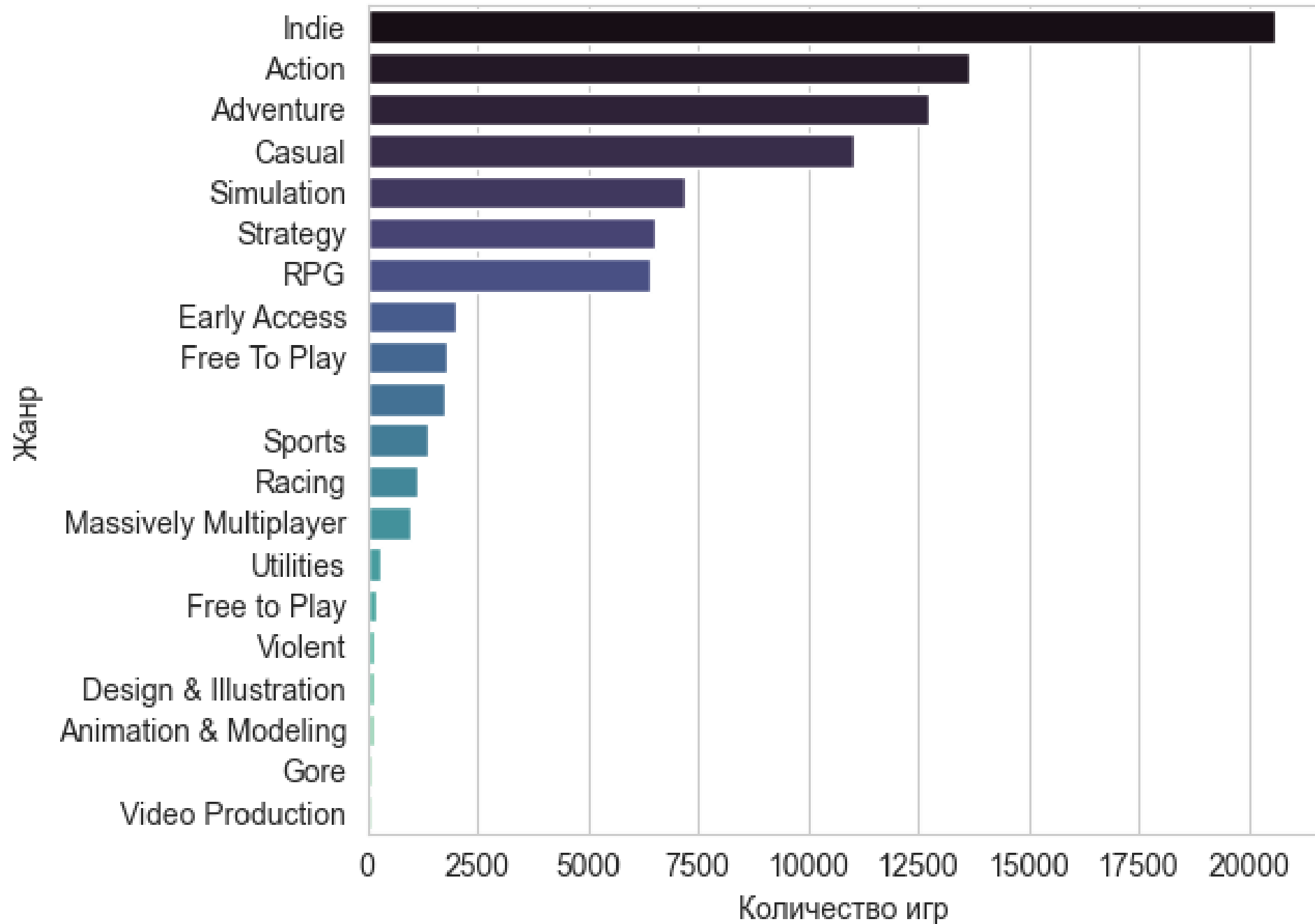
Название игры



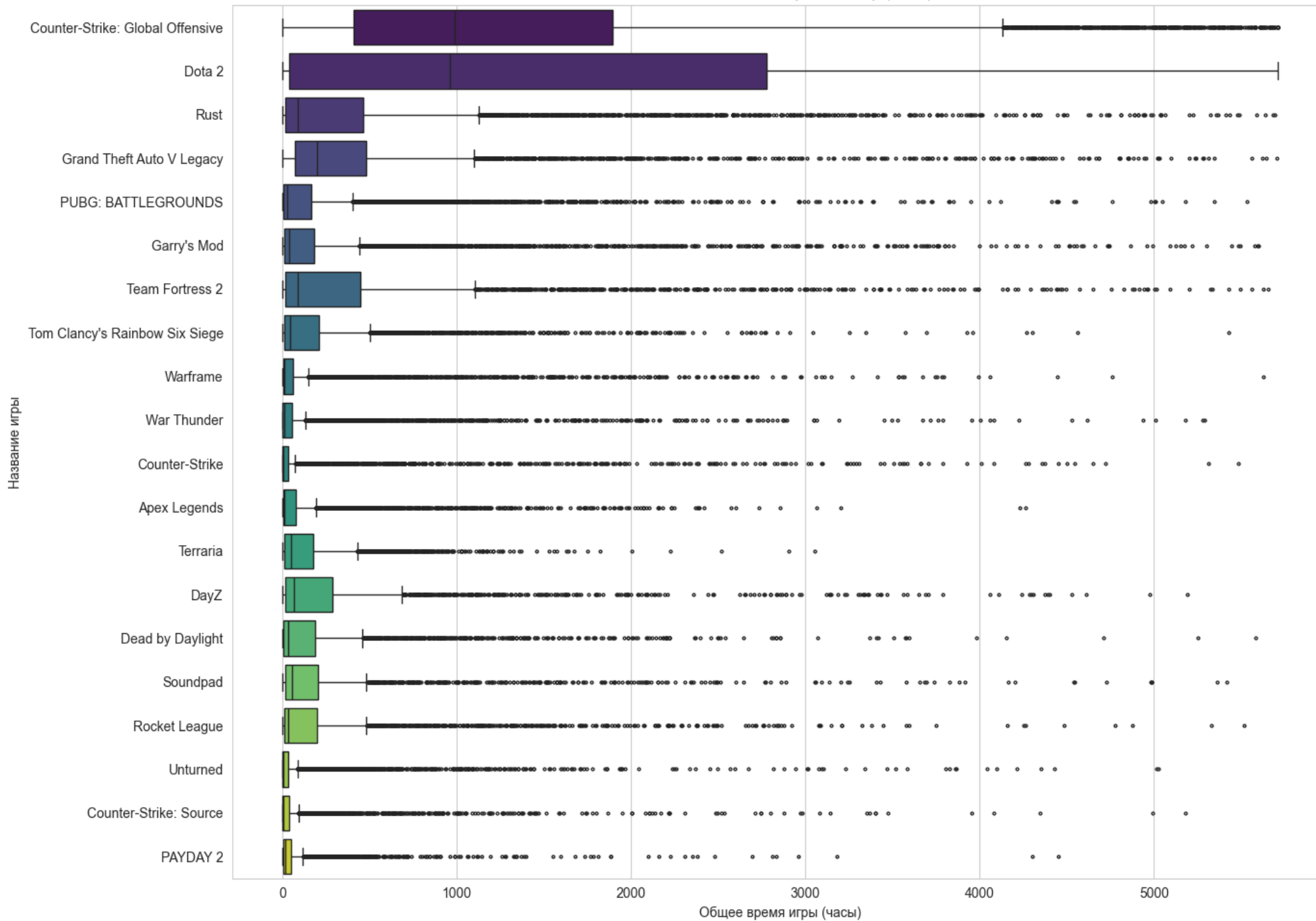
Распределение общего времени игры (в часах) у публичных пользователей



Топ-20 жанров в steamGamesDataset

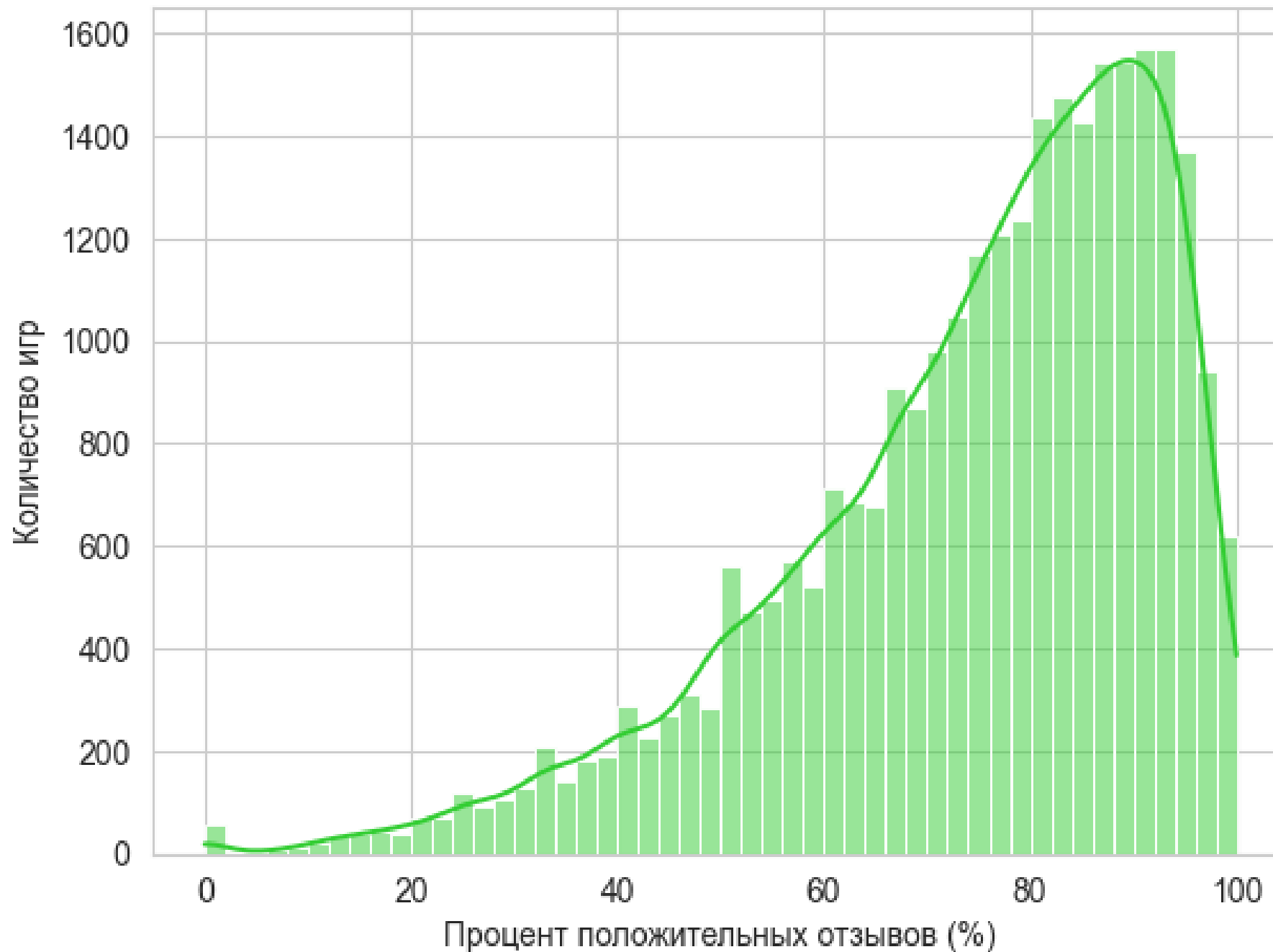


Box Plot: Топ самых играемых игр (часы)

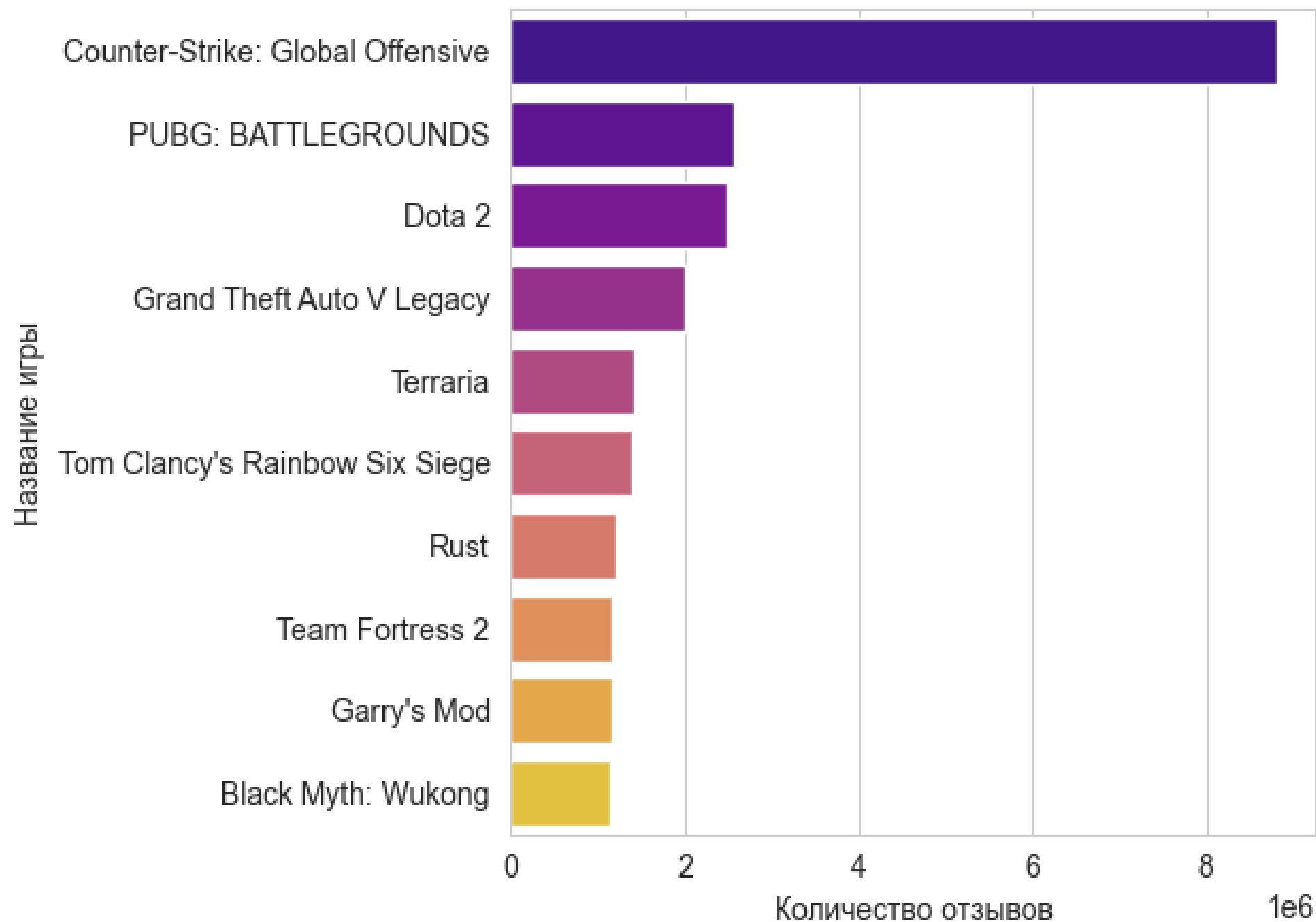


300
ЛЕТ
СПбГУ

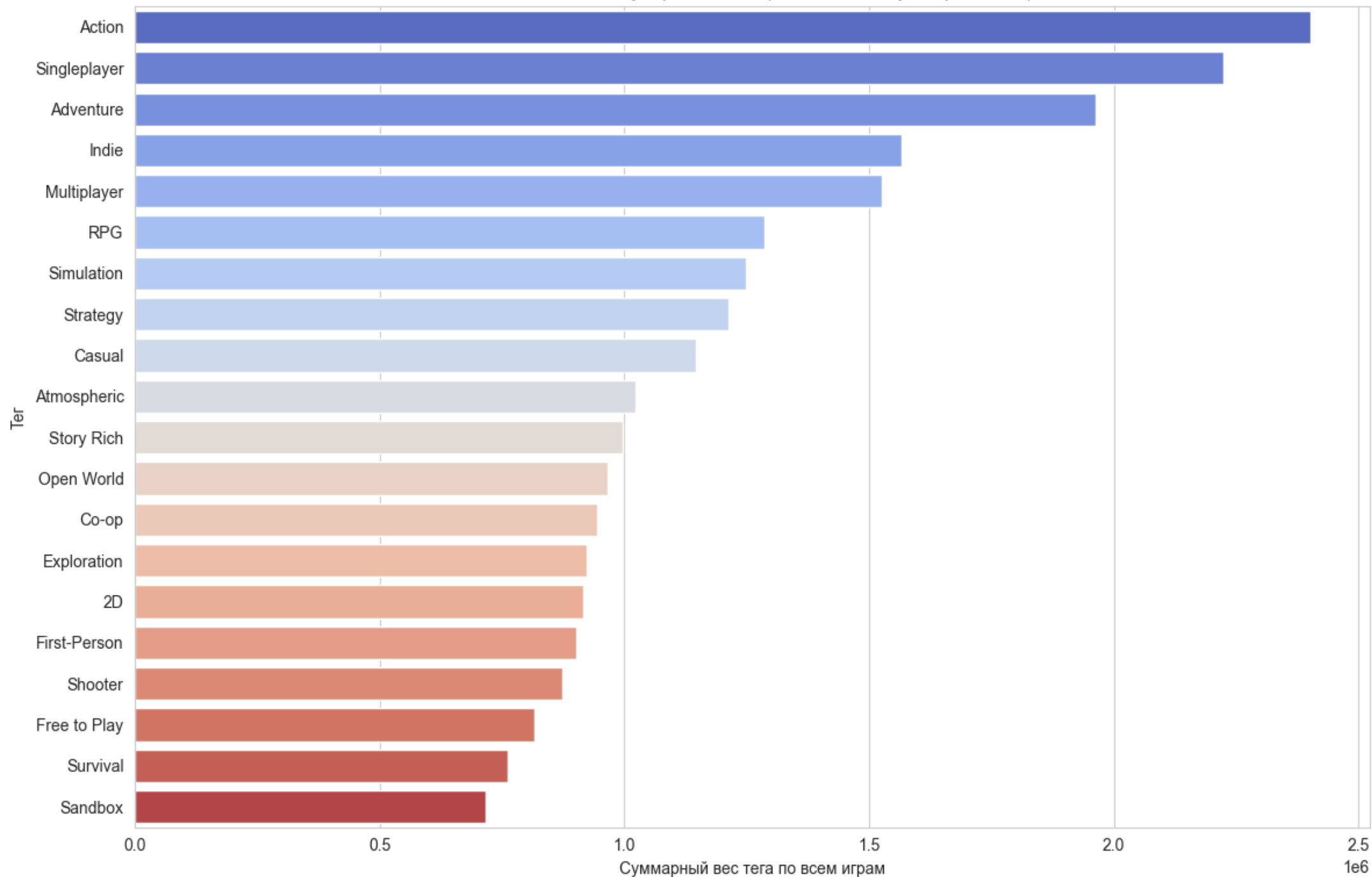
Распределение процента положительных отзывов по играм



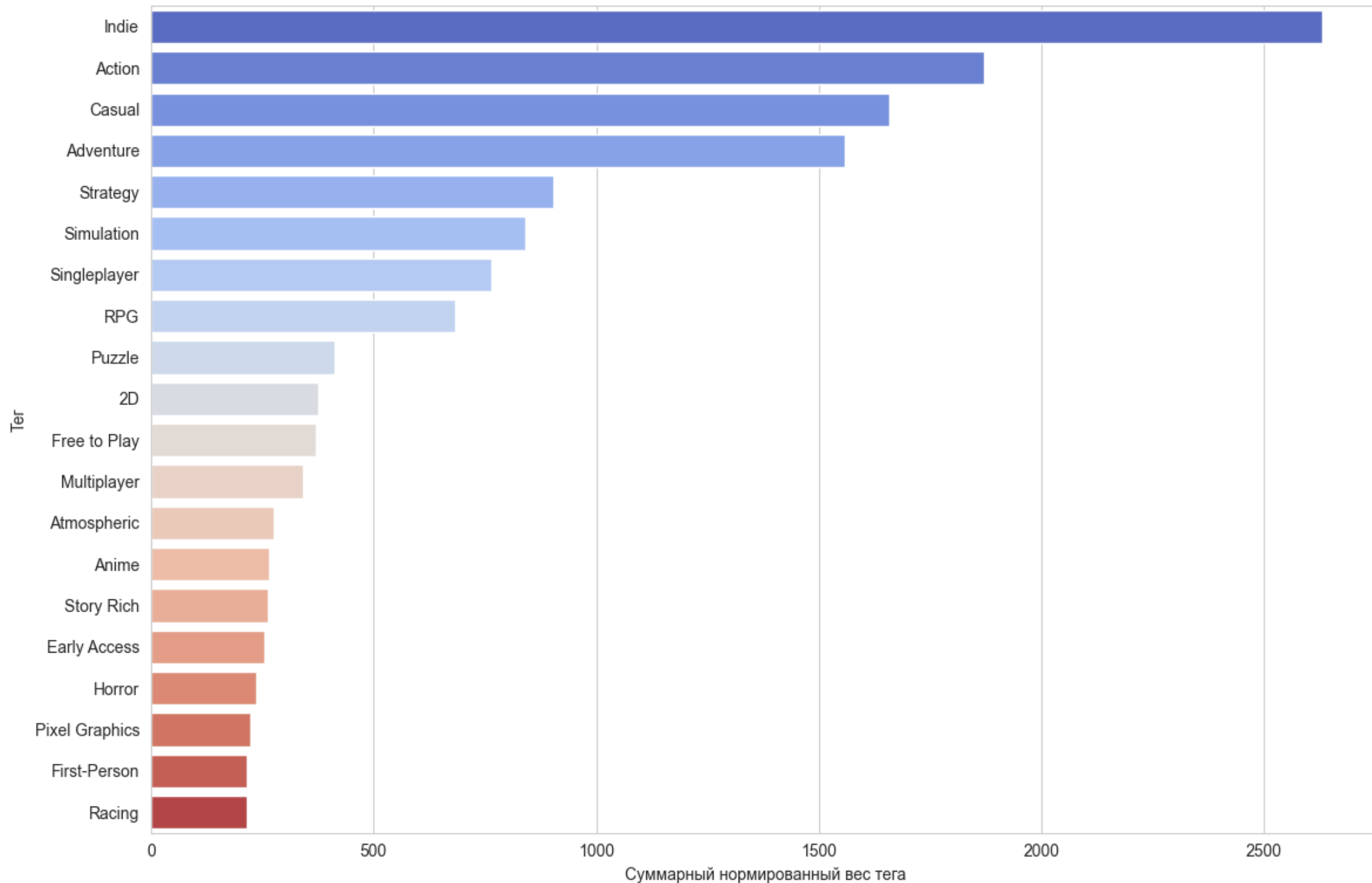
Топ-10 игр по общему количеству отзывов



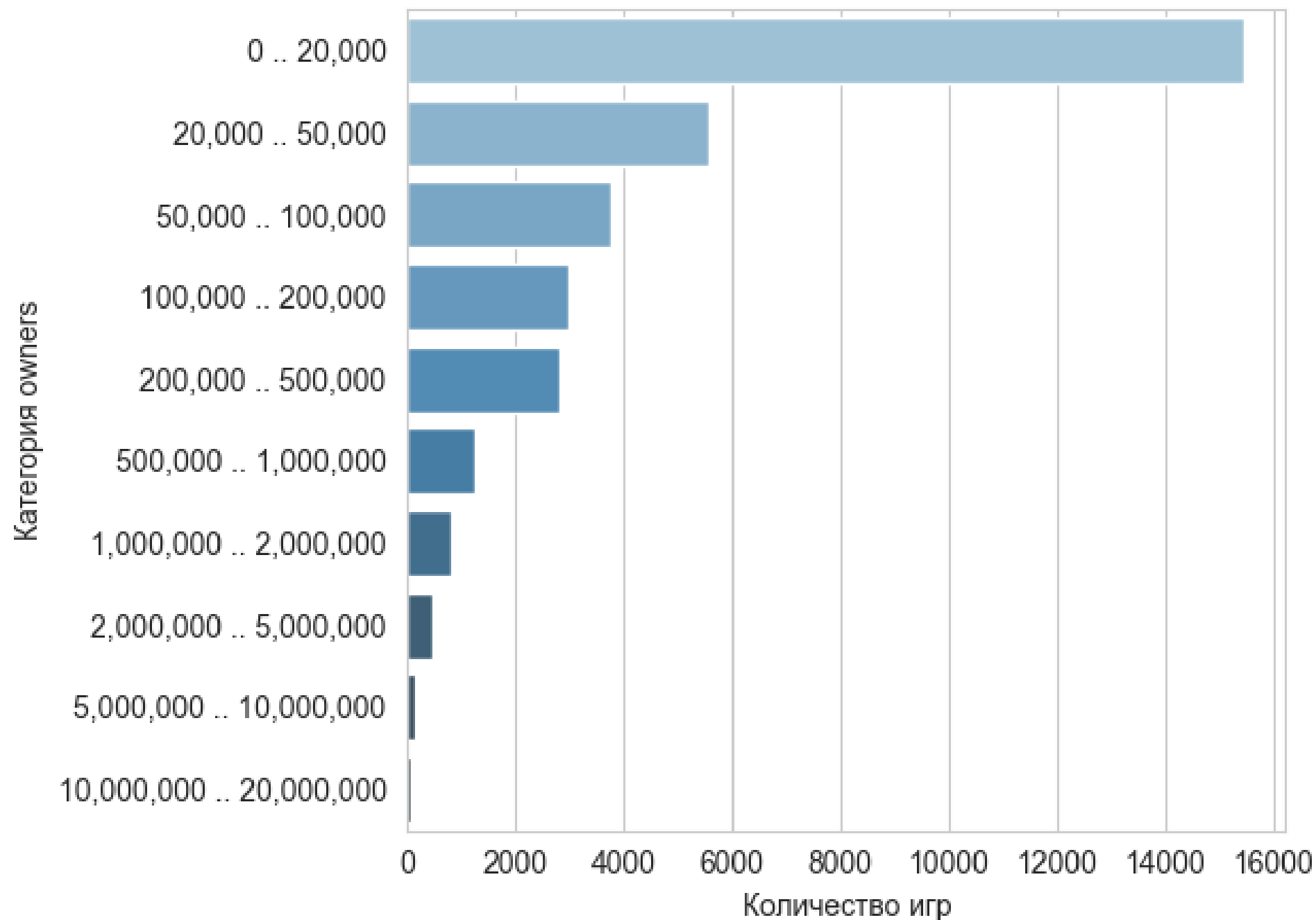
Топ-20 самых популярных тегов (абсолютный суммарный вес)



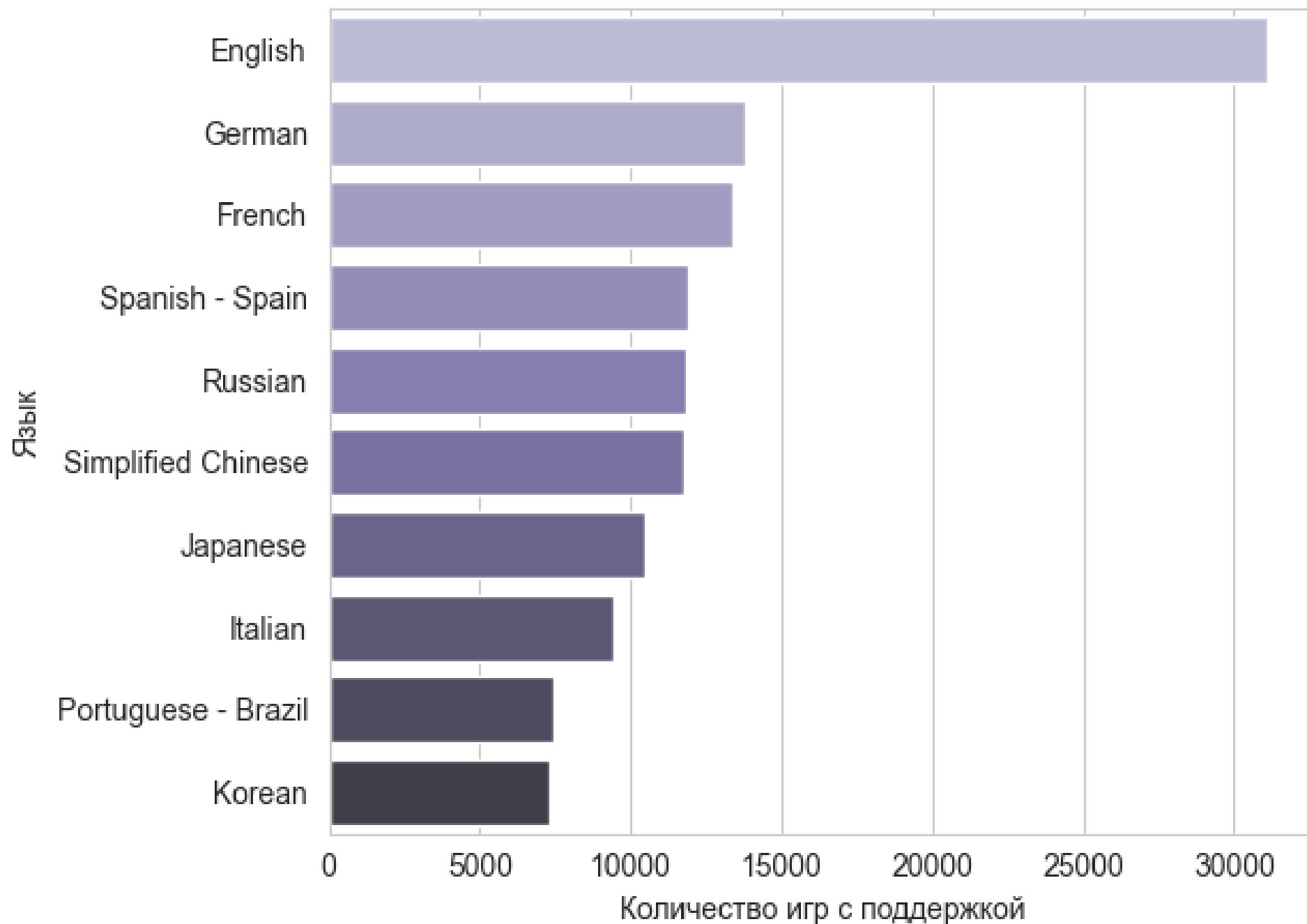
Топ-20 самых популярных тегов (нормированный вес внутри каждой игры)



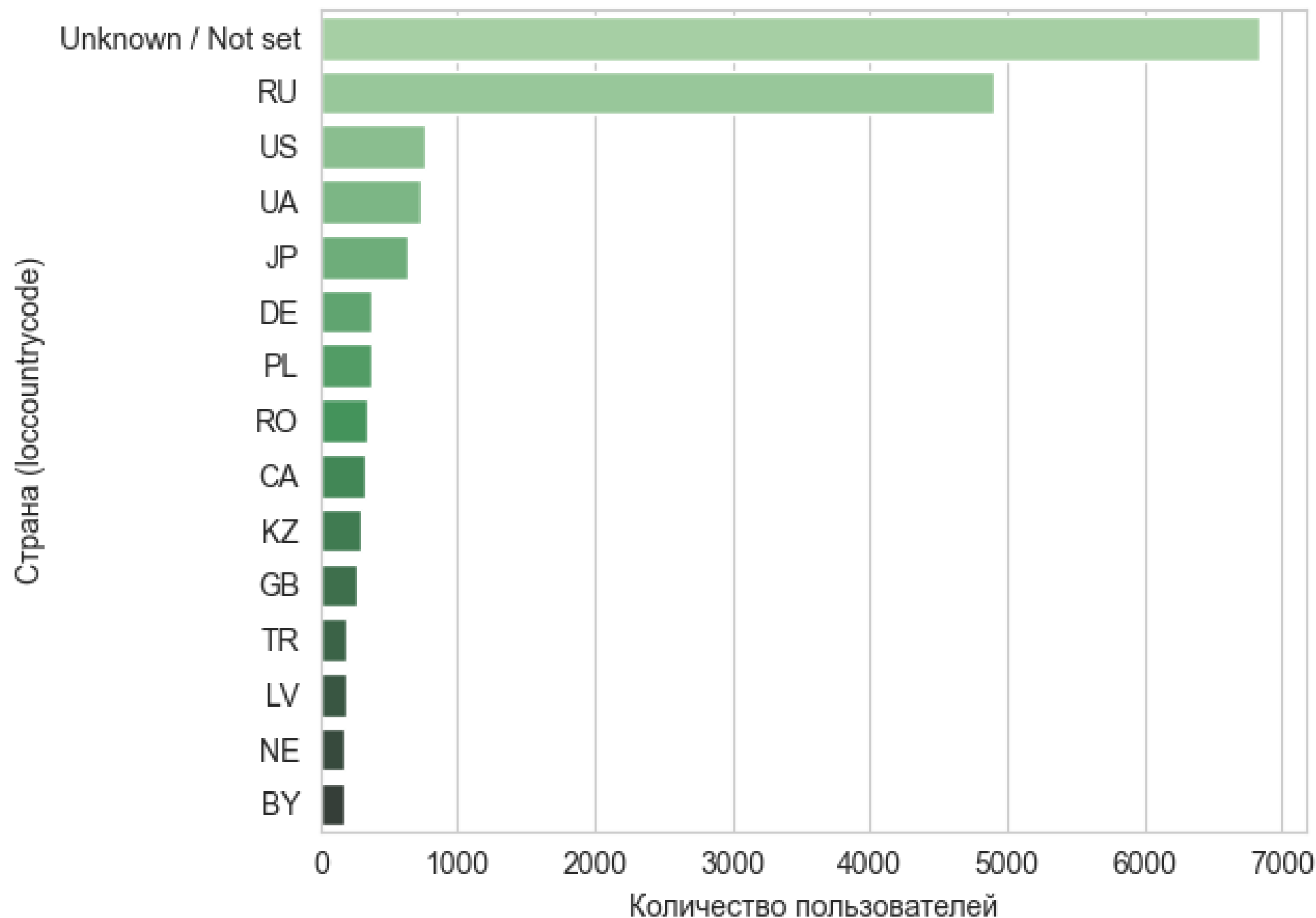
Топ-10 категорий по количеству владельцев игр



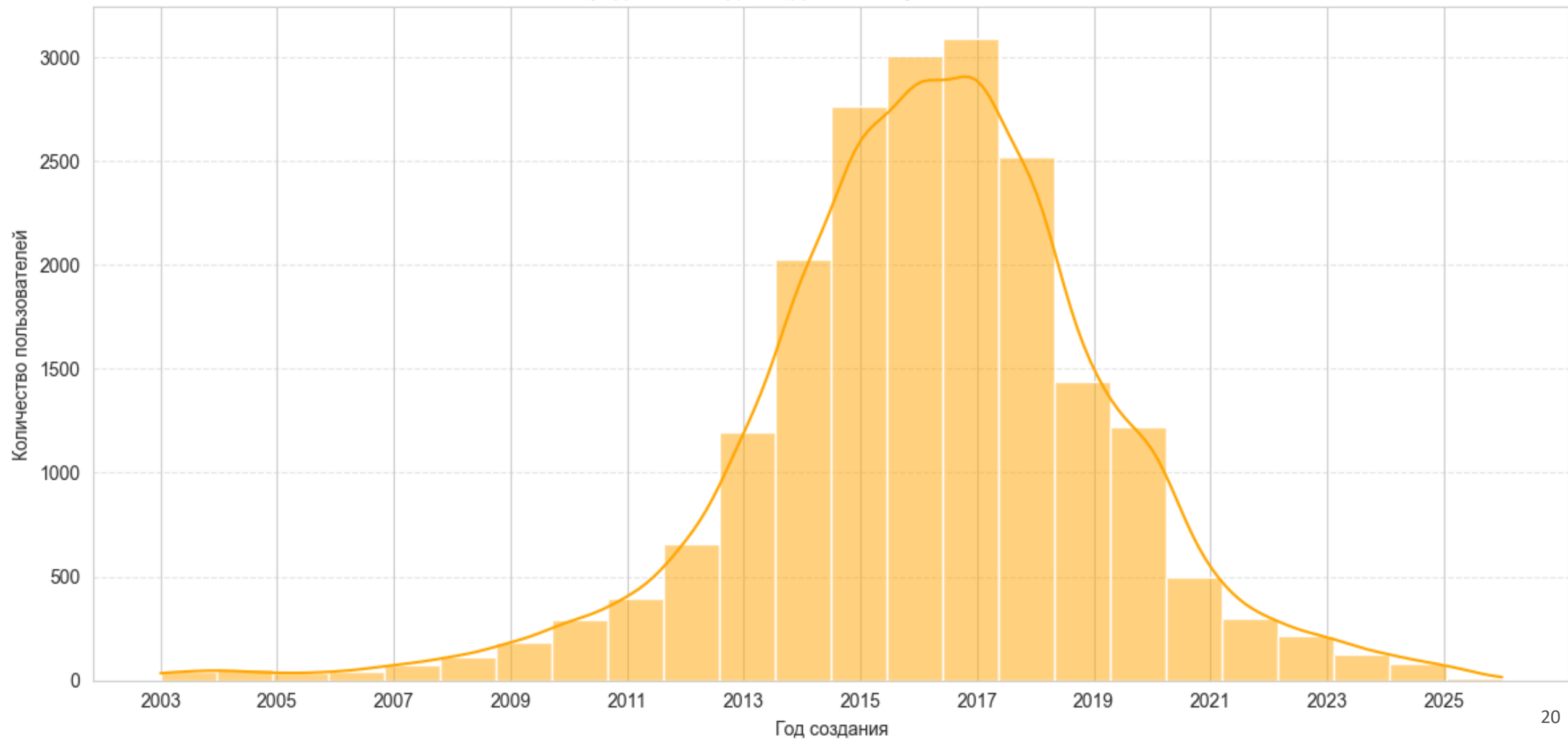
Топ-10 самых поддерживаемых языков



Топ-15 стран пользователей (публичные профили)



Распределение года создания аккаунтов пользователей



Кластеризация пользователей по стилю игры (K-Means)

Для сегментации пользователей было построено **4 признака**, отражающих игровое поведение:

- Количество игр в библиотеке
- Общее наигранное время (в часах)
- Среднее время игры на одну игру
- Доля «долгих» игр (более 50 часов)

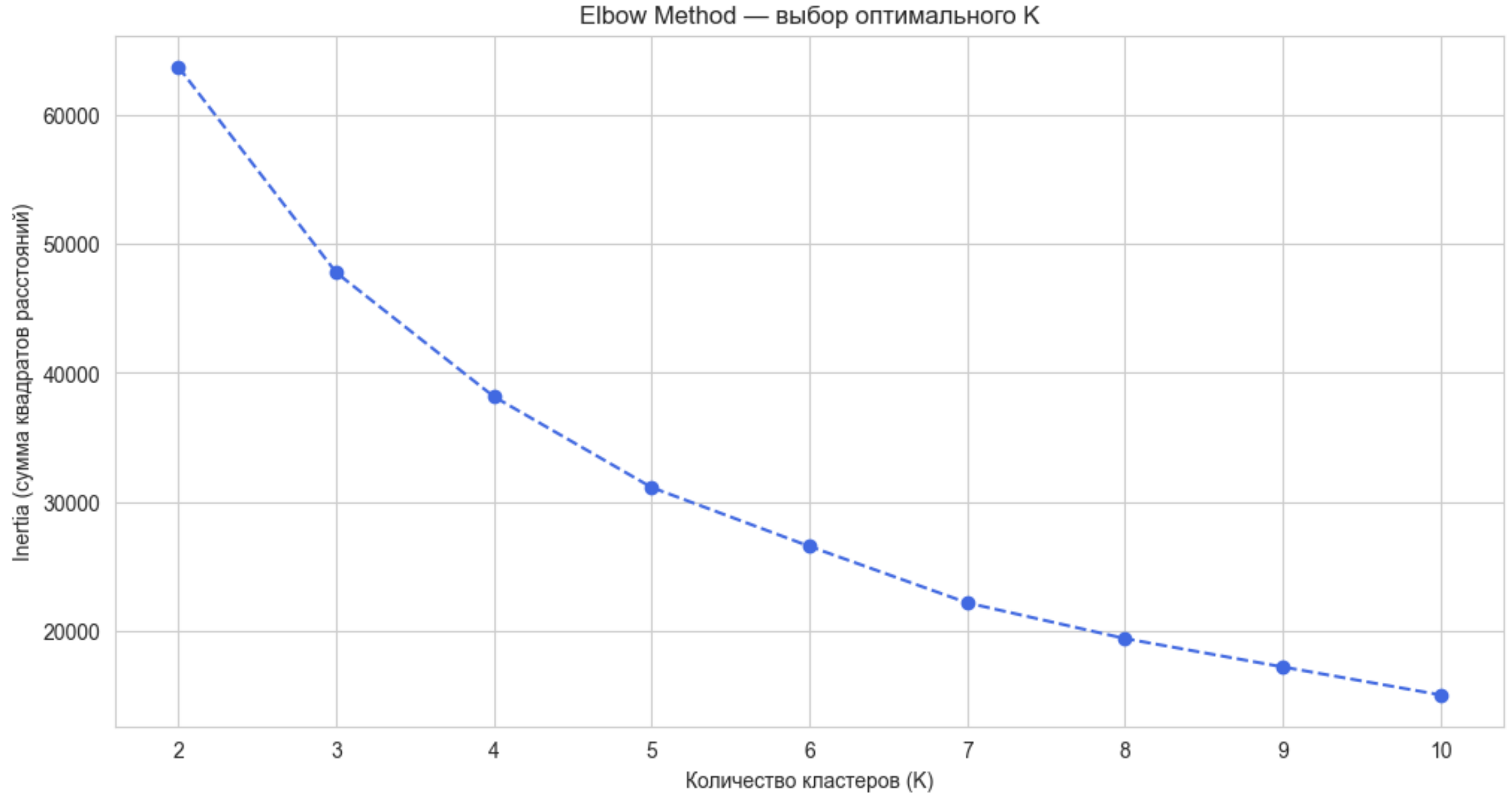
Признаки были **масштабированы** с помощью `StandardScaler`, чтобы все переменные имели одинаковый вес.

Использовался метод «локтя» (Elbow Method).
Оптимальным было выбрано **K = 4 кластера**

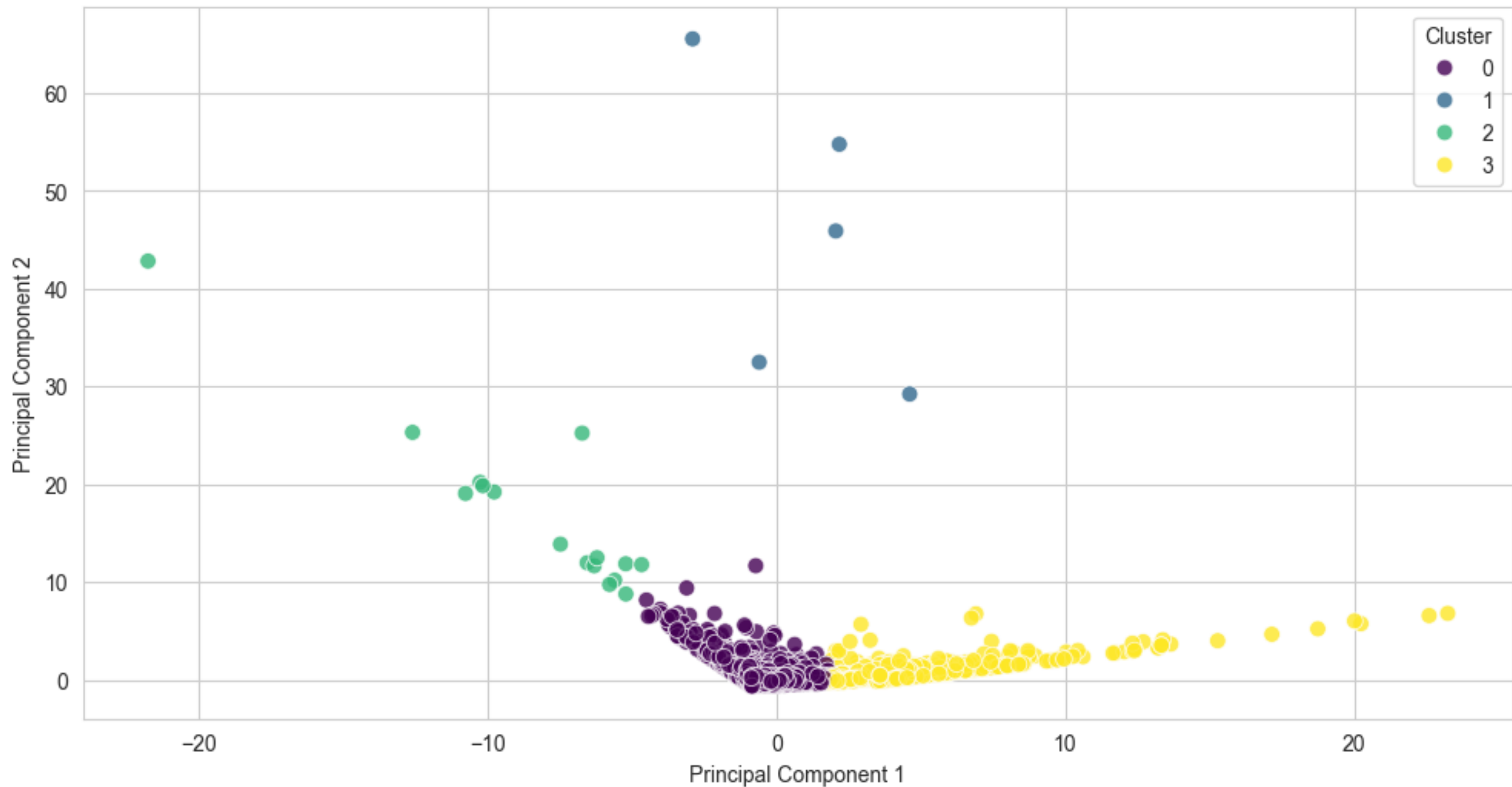
Результаты кластеризации:

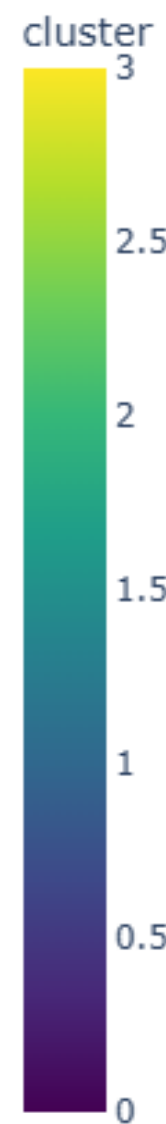
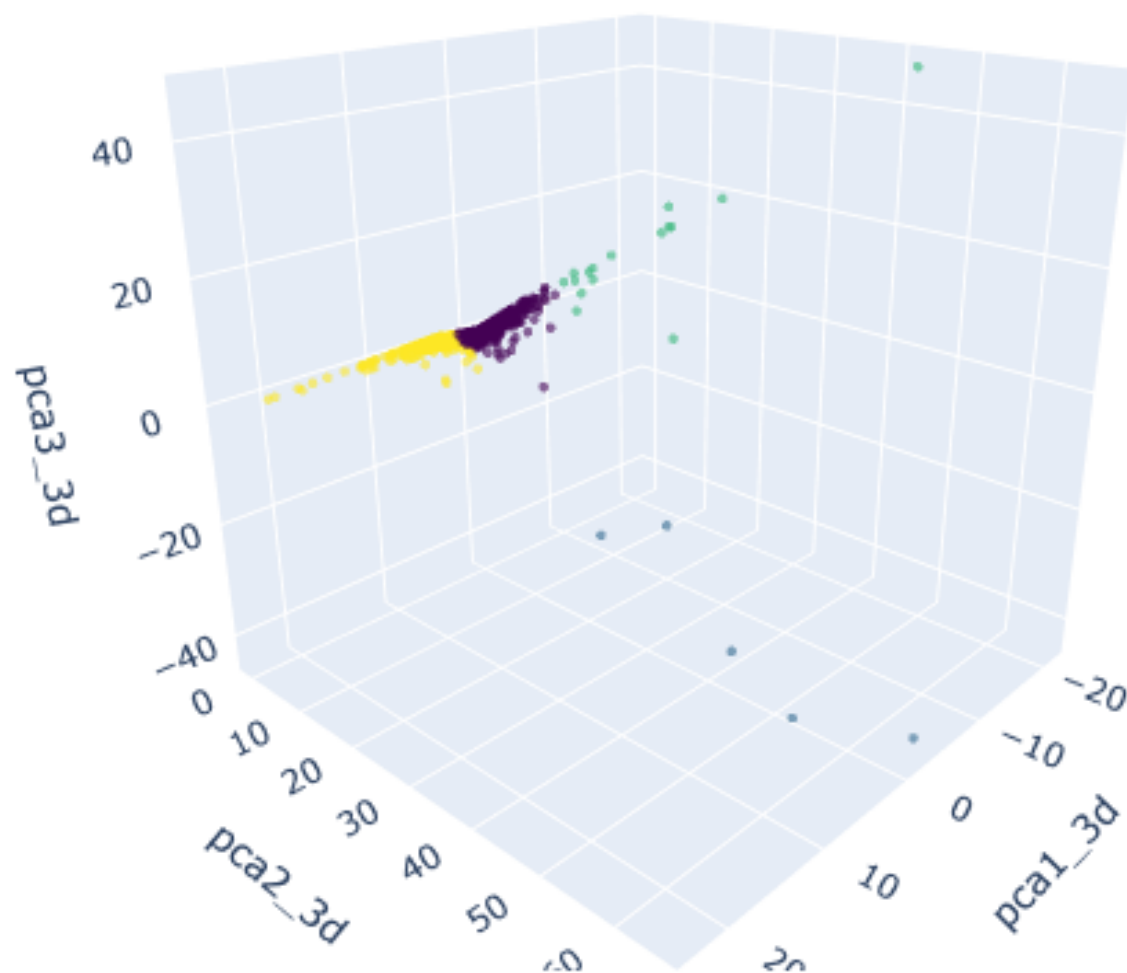
Кластер	Пользователей
0	19 222
3	1 147
2	16
1	5

Метод «локтя» по стилю игры (Elbow Method)

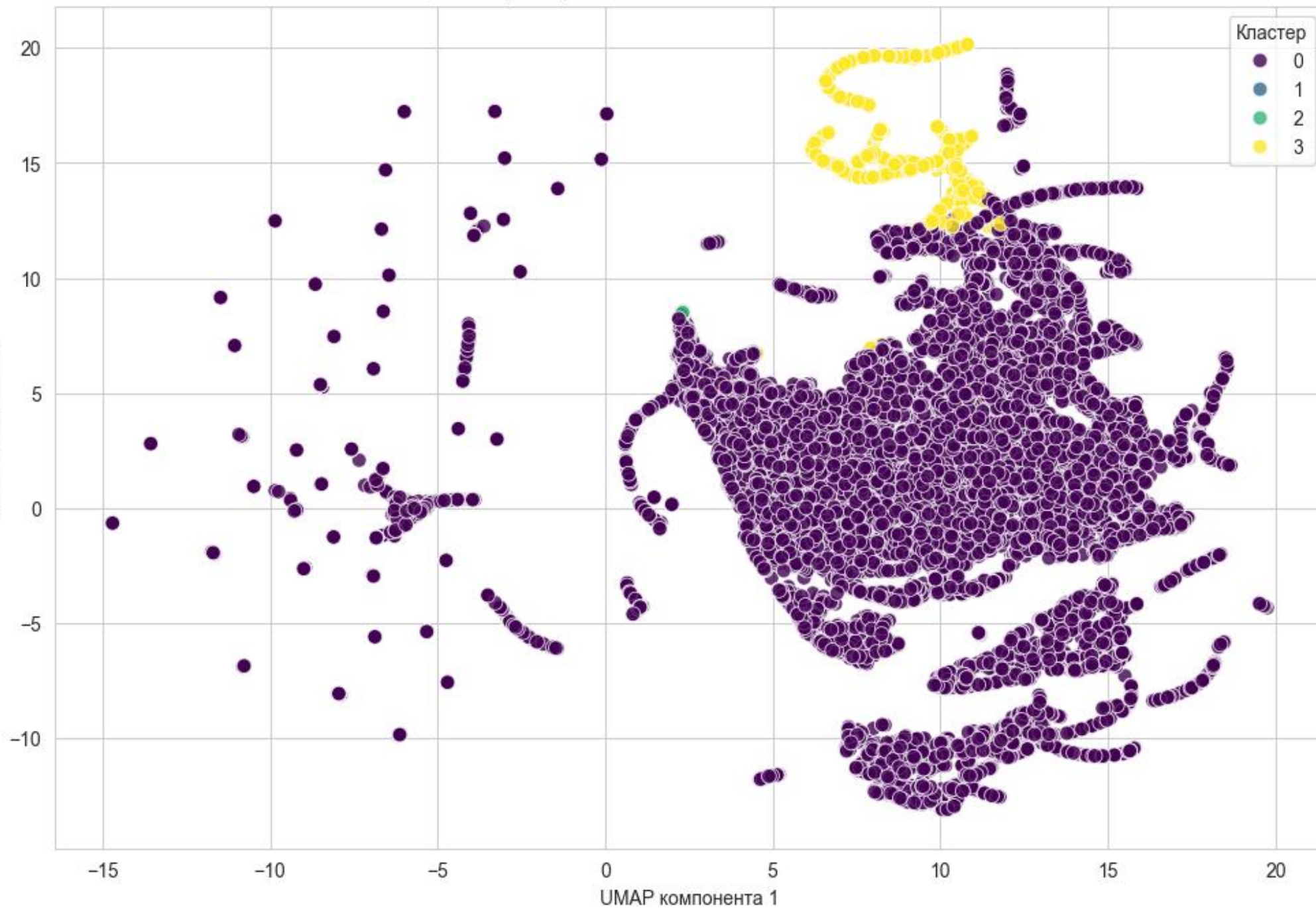


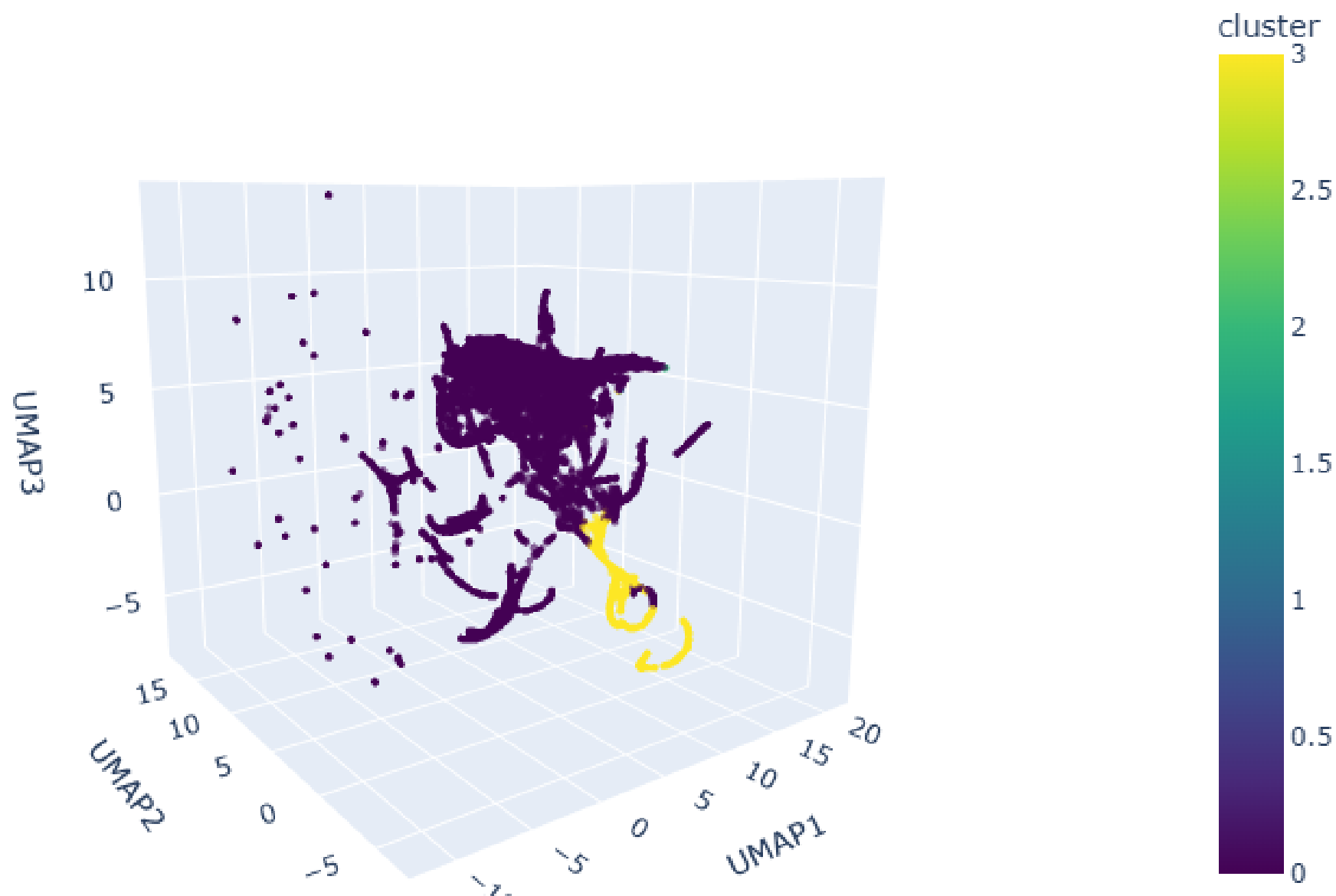
Кластеризация пользователей Steam - PCA 2D



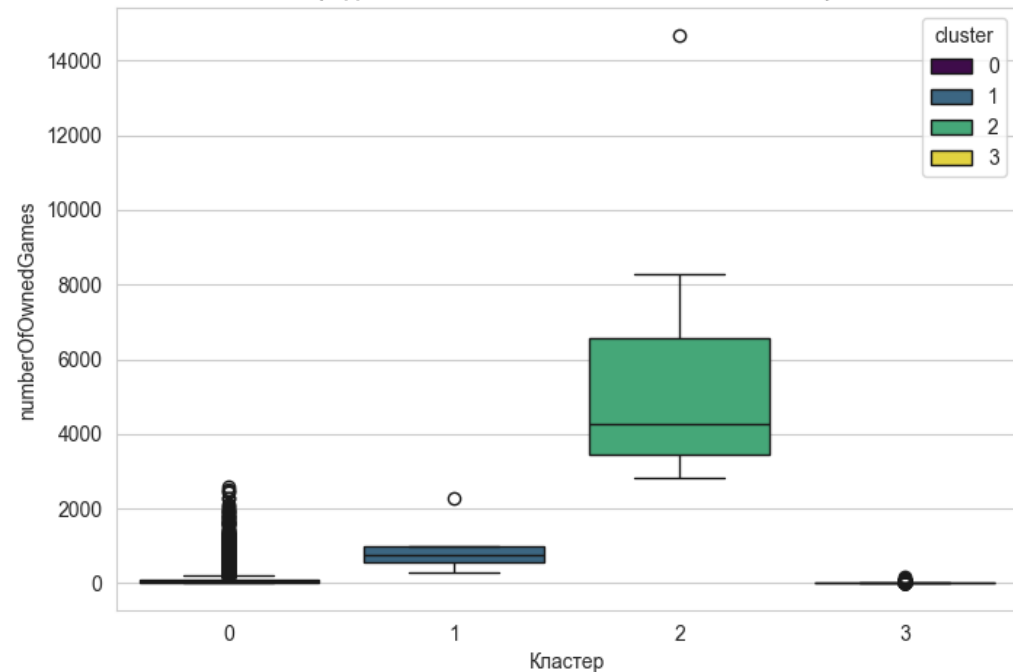


UMAP компонента 2

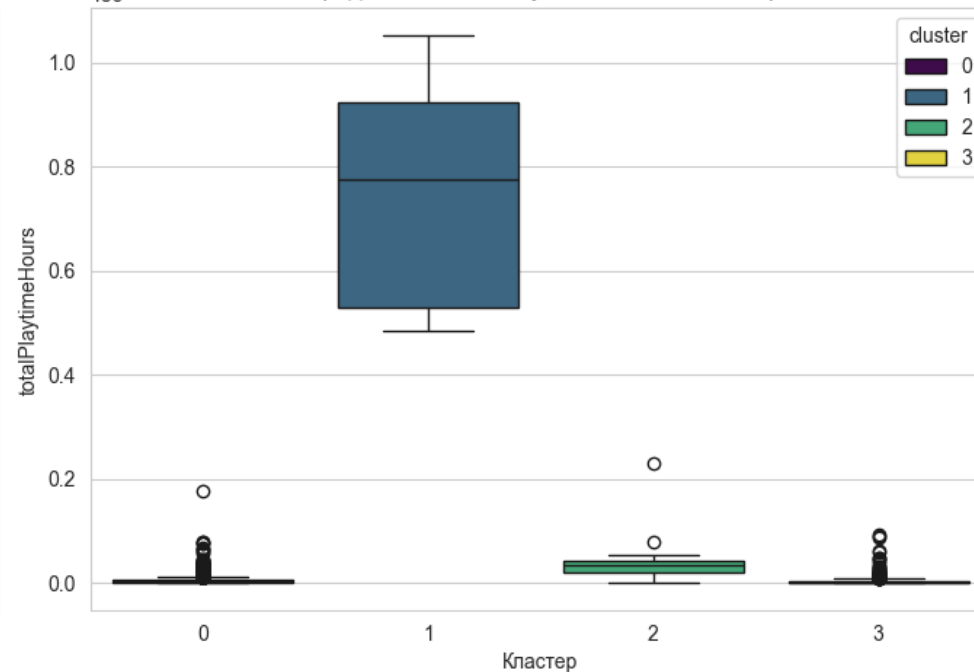




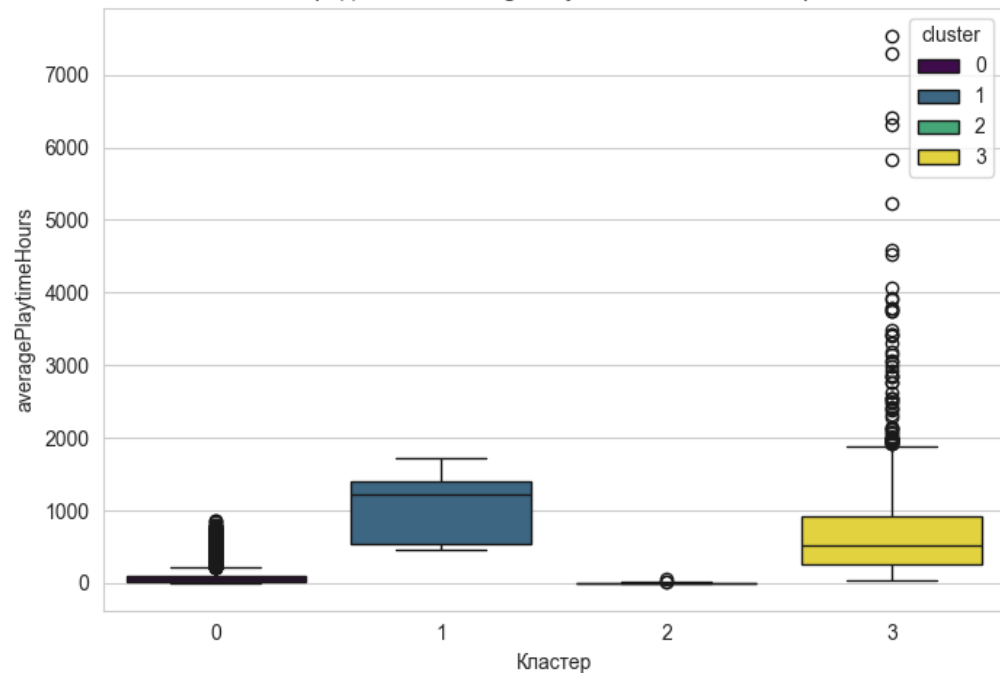
Распределение numberOfOwnedGames по кластерам



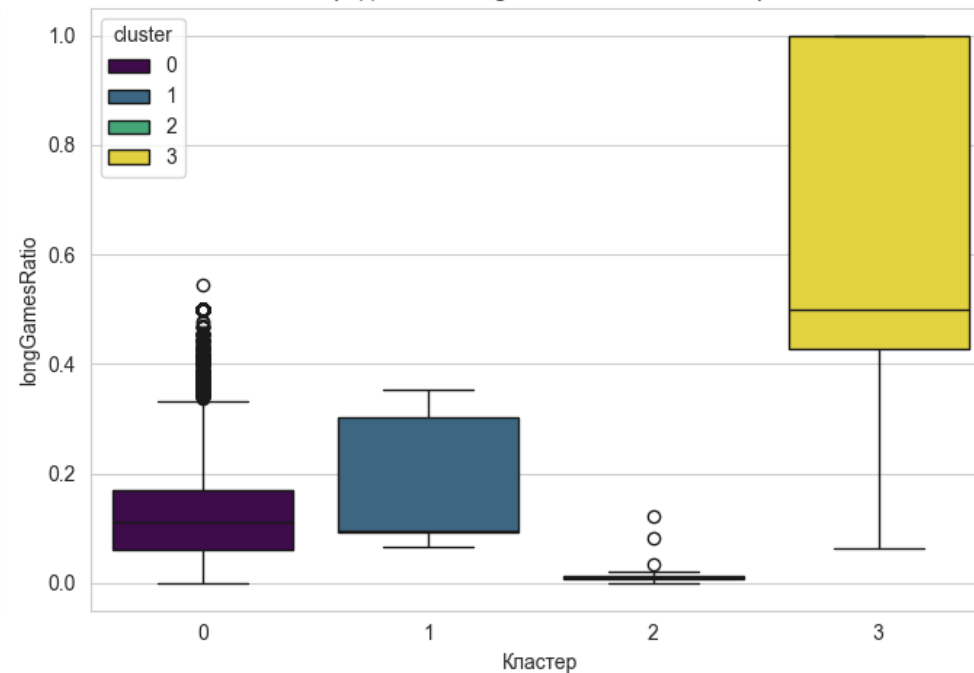
Распределение totalPlaytimeHours по кластерам



Распределение averagePlaytimeHours по кластерам



Распределение longGamesRatio по кластерам



Кластеризация по игровым предпочтениям (теги)

Вторая кластеризация проводилась на основе **жанровых предпочтений** пользователей.

Признаки:

- Для каждого пользователя рассчитана **нормализованная доля** каждого тега в его библиотеке
- Использовано более 300 тегов (Action, RPG, Strategy, Indie и др.)

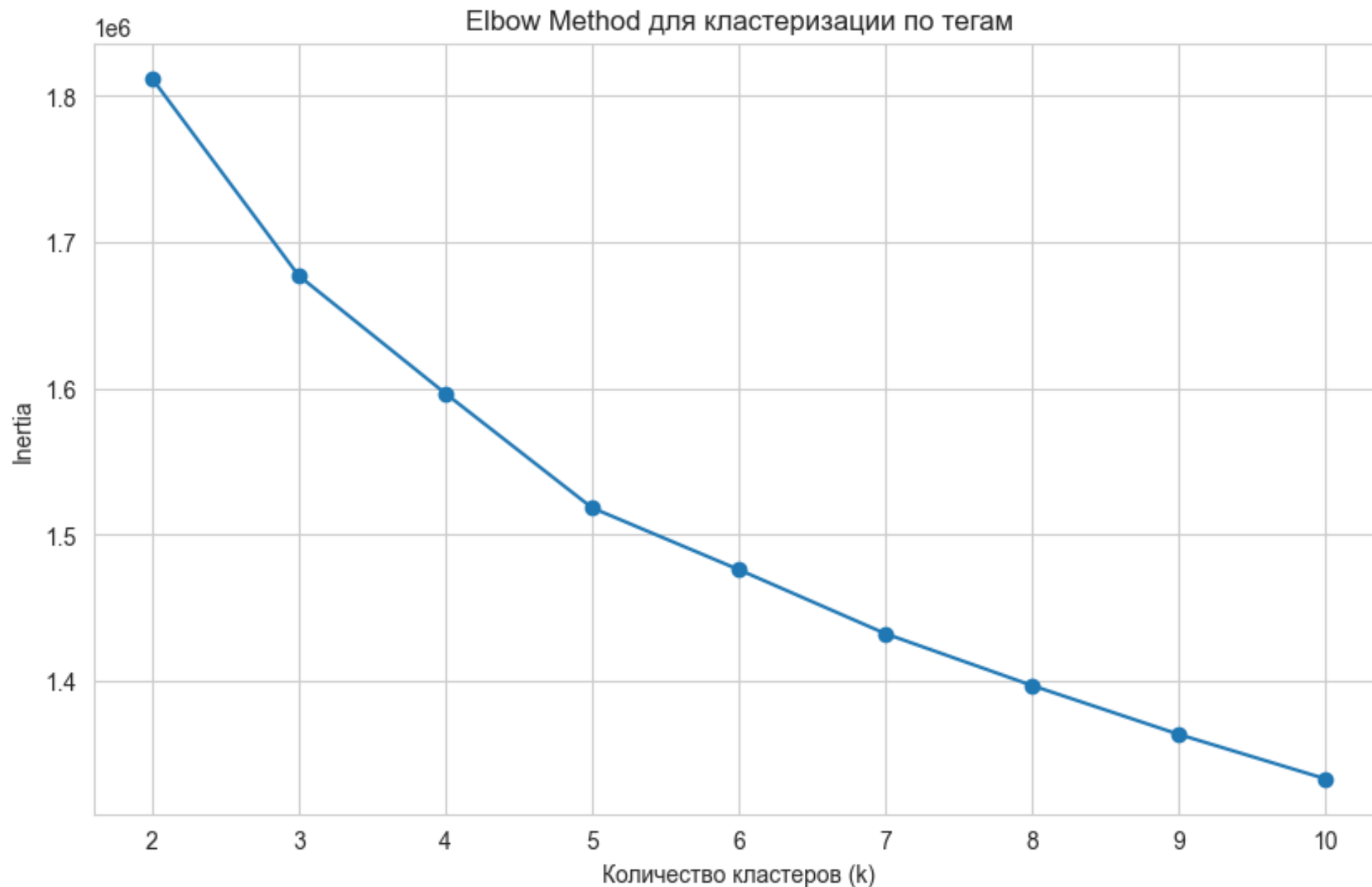
Метод:

- K-Means кластеризация
- Количество кластеров: **K = 8**

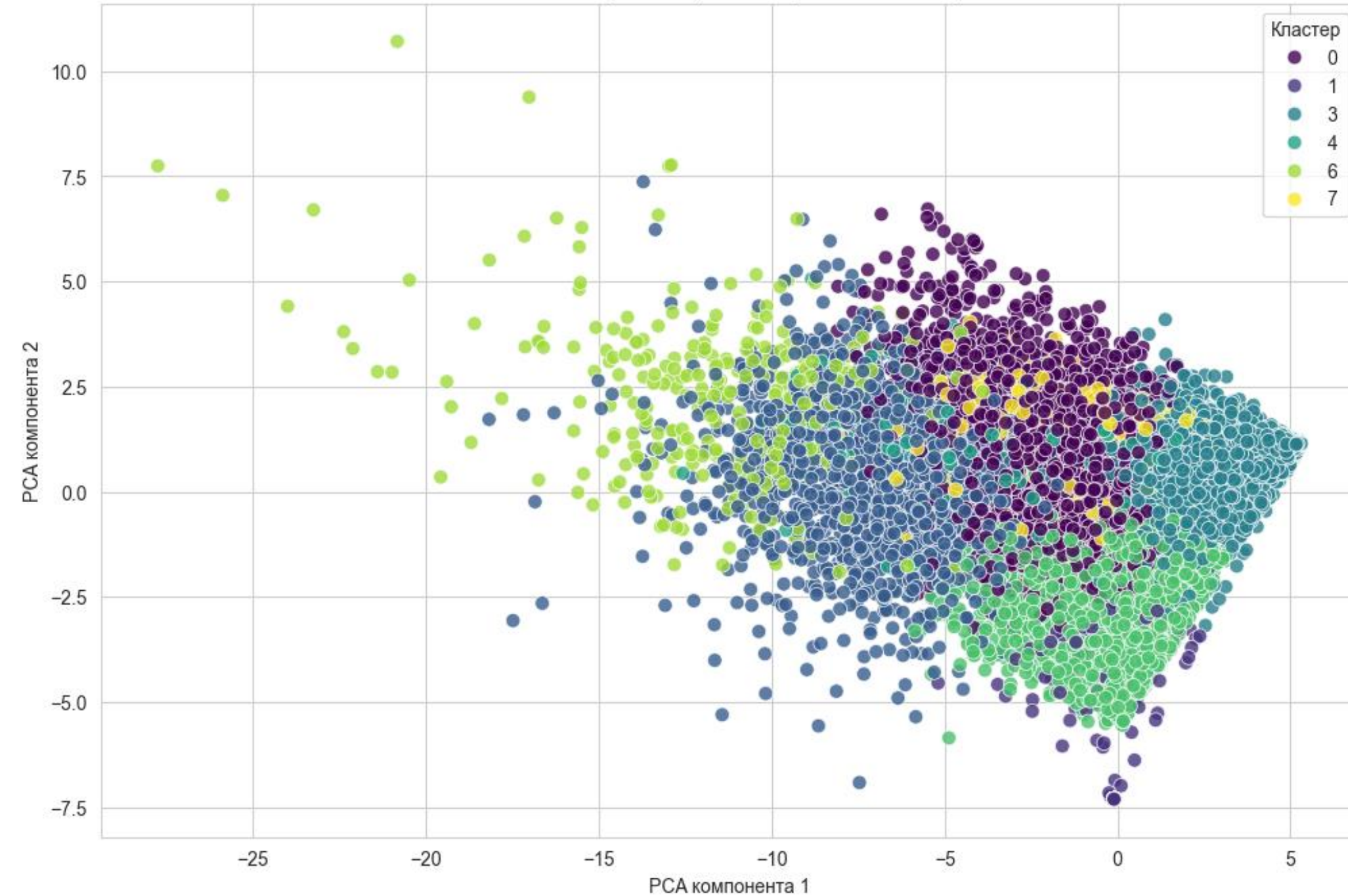
Результаты кластеризации:

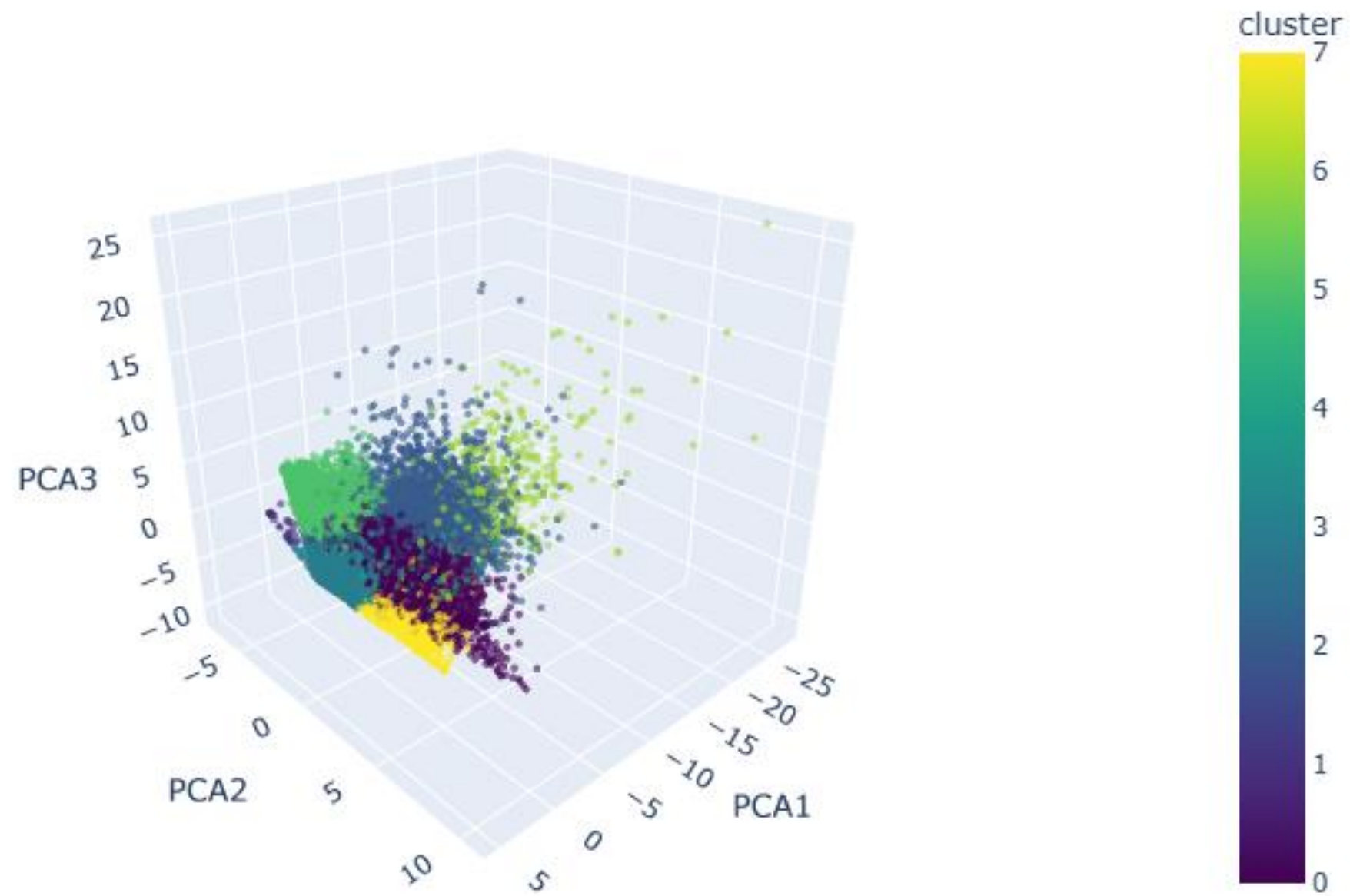
Кластер	Пользователей
3	8375
0	4460
2	2172
1	1834
5	1779
7	843
4	534
6	393

Метод «локтя» по тегам (Elbow Method)

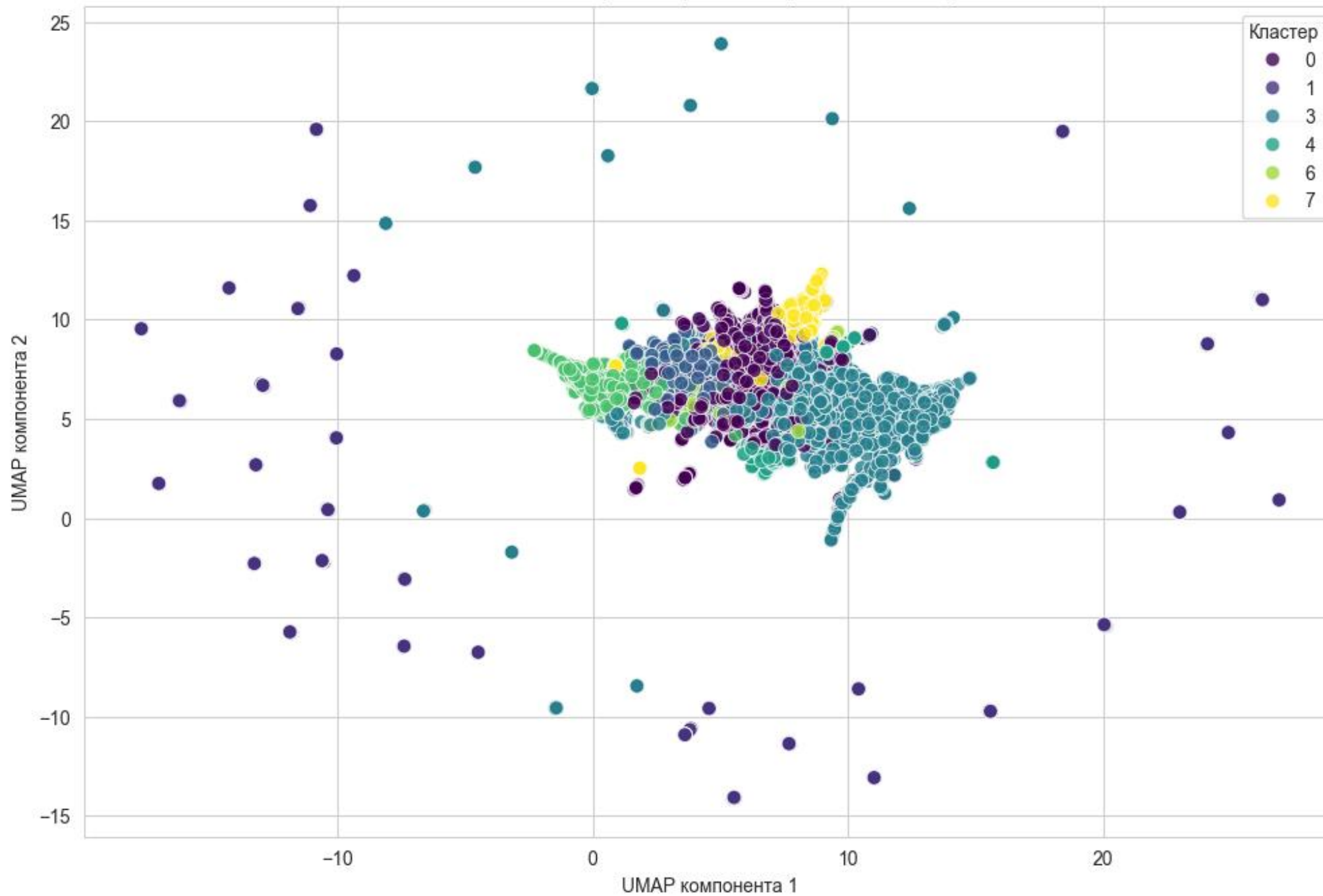


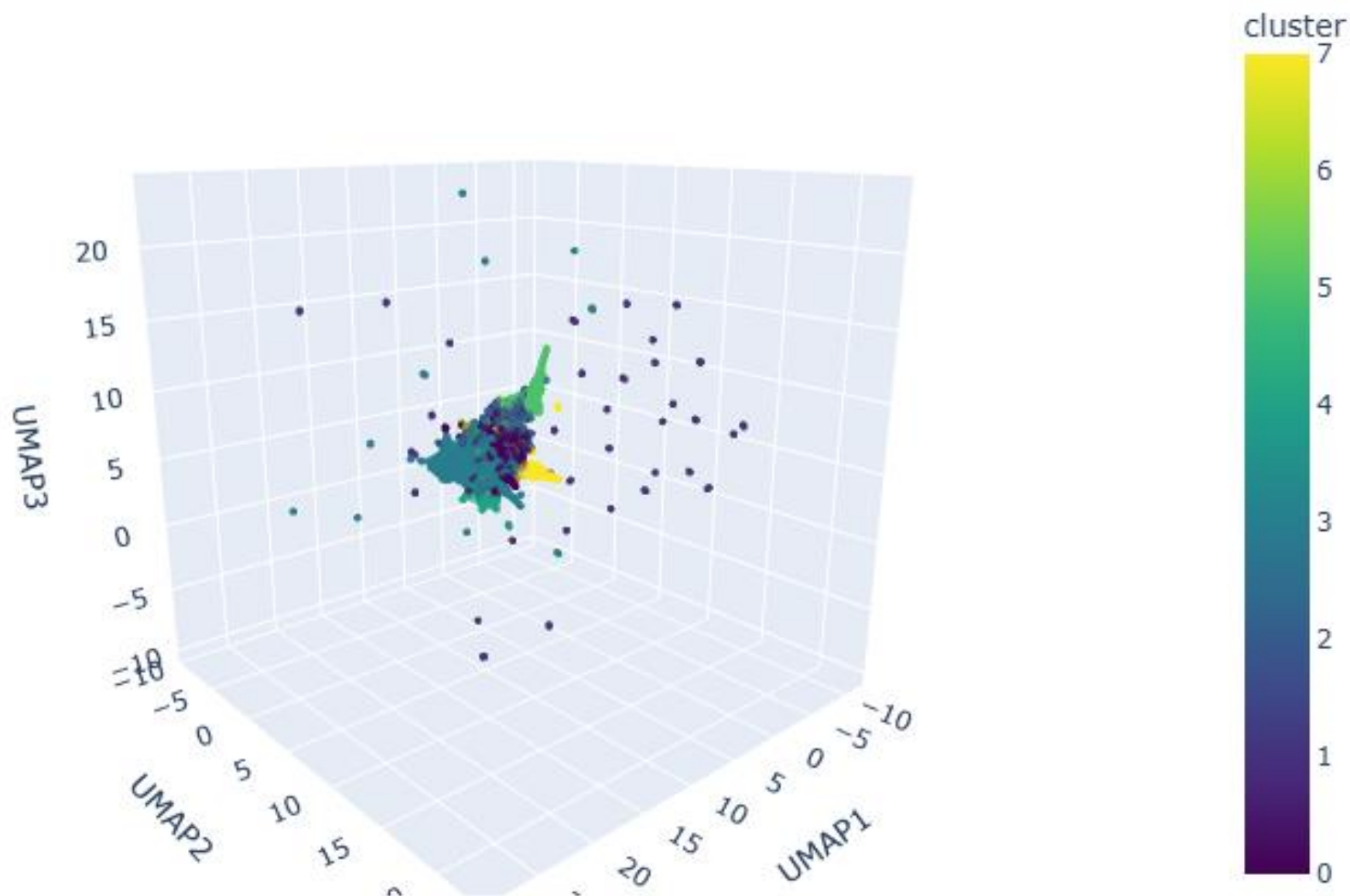
2D PCA визуализация кластеров по тегам игр



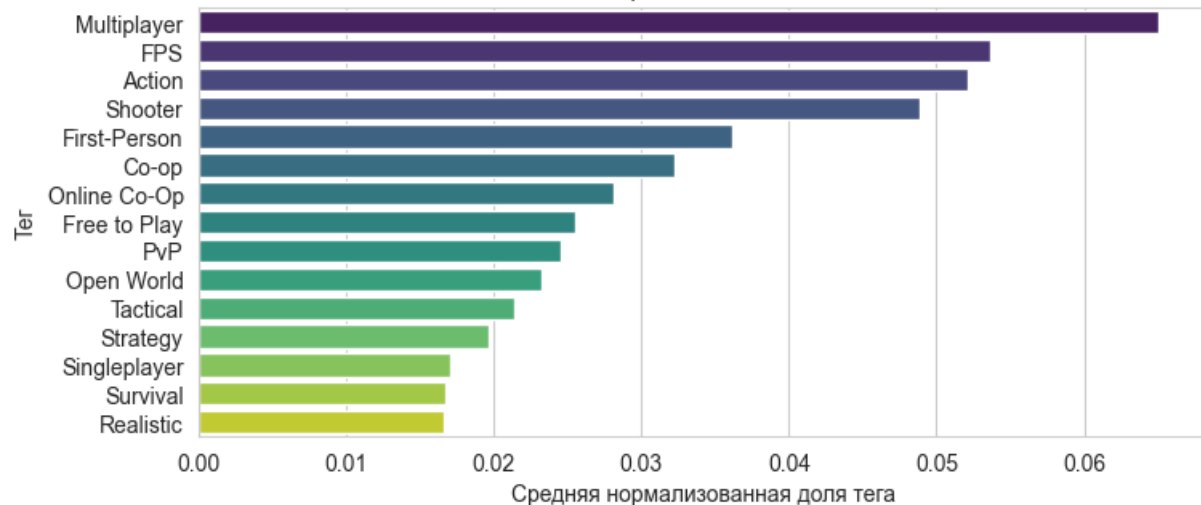


2D UMAP визуализация кластеров по тегам игр

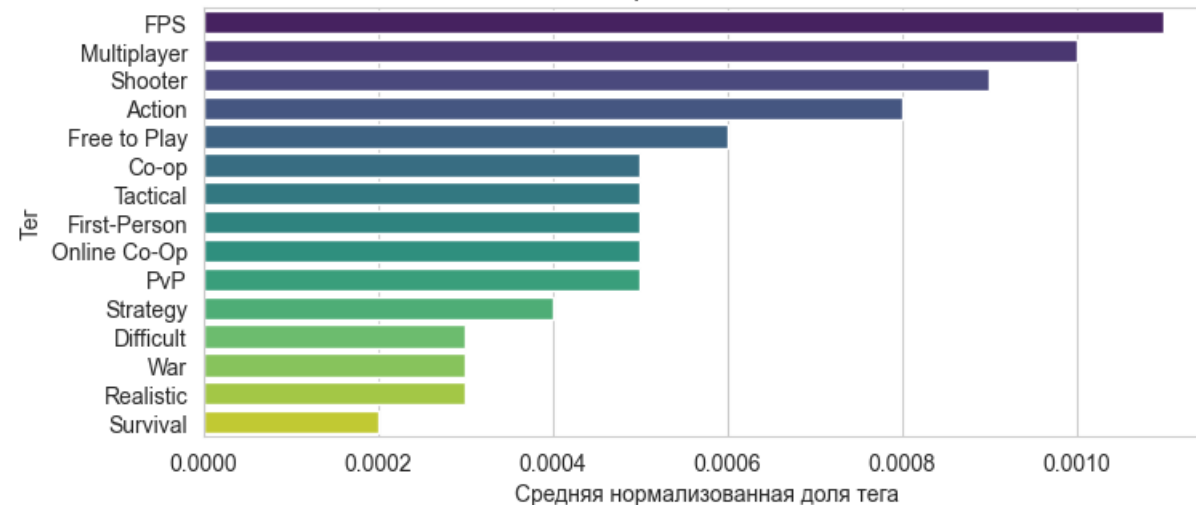




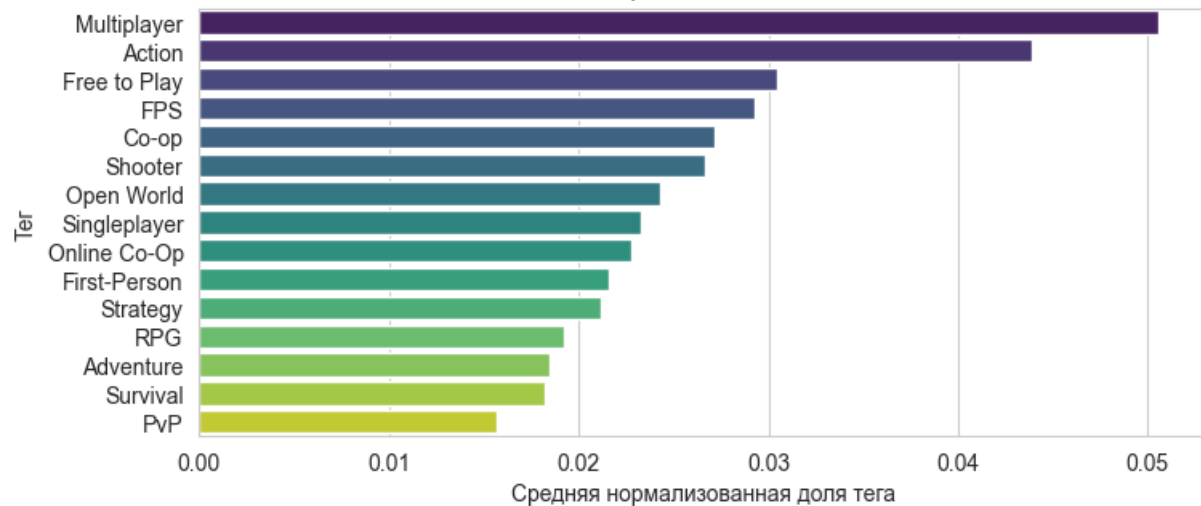
Кластер 0 — Топ-15 тегов



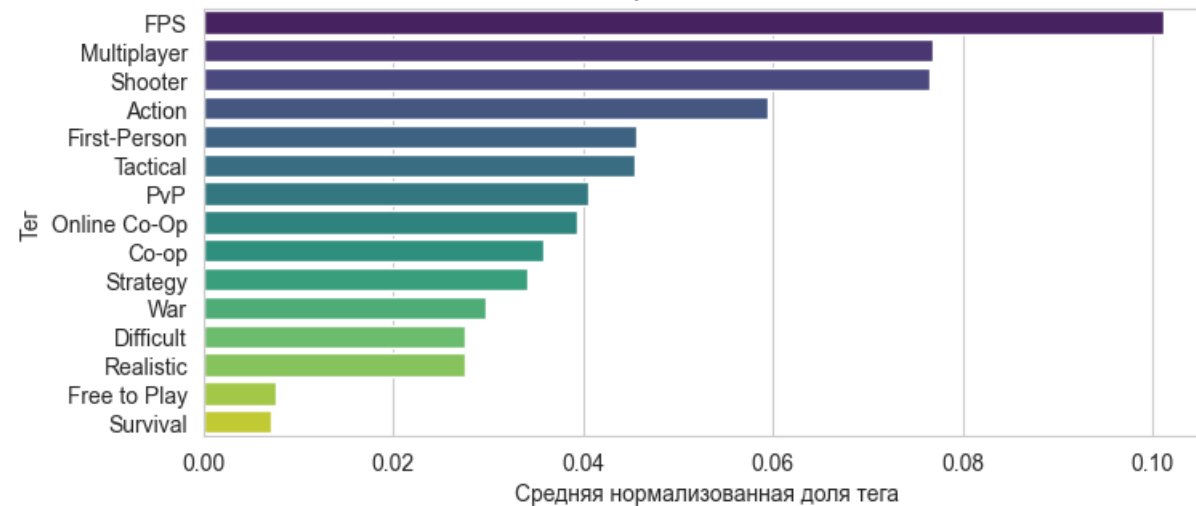
Кластер 1 — Топ-15 тегов



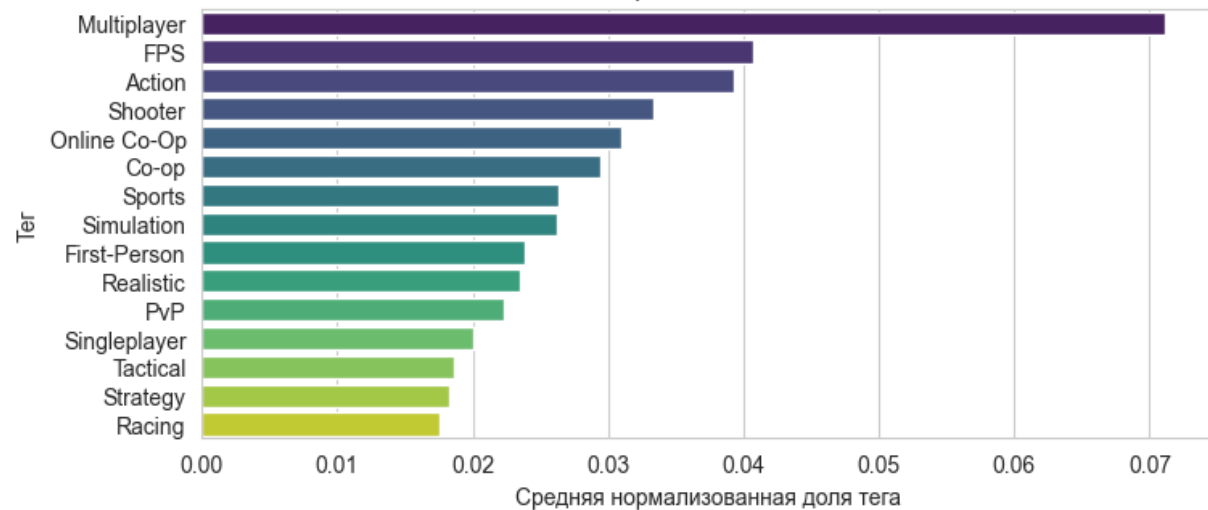
Кластер 2 — Топ-15 тегов



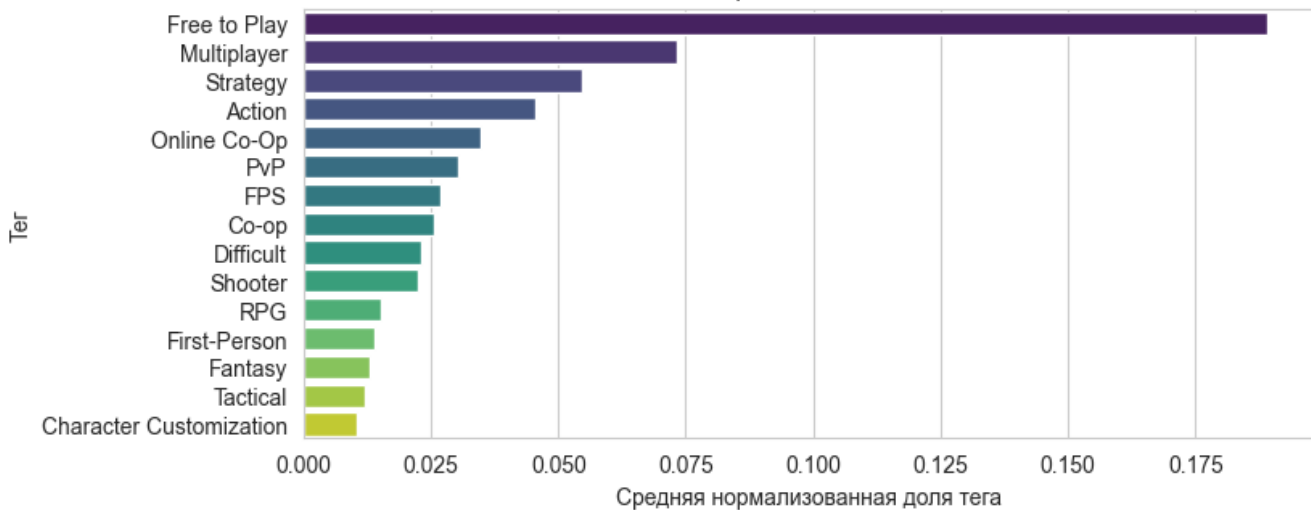
Кластер 3 — Топ-15 тегов



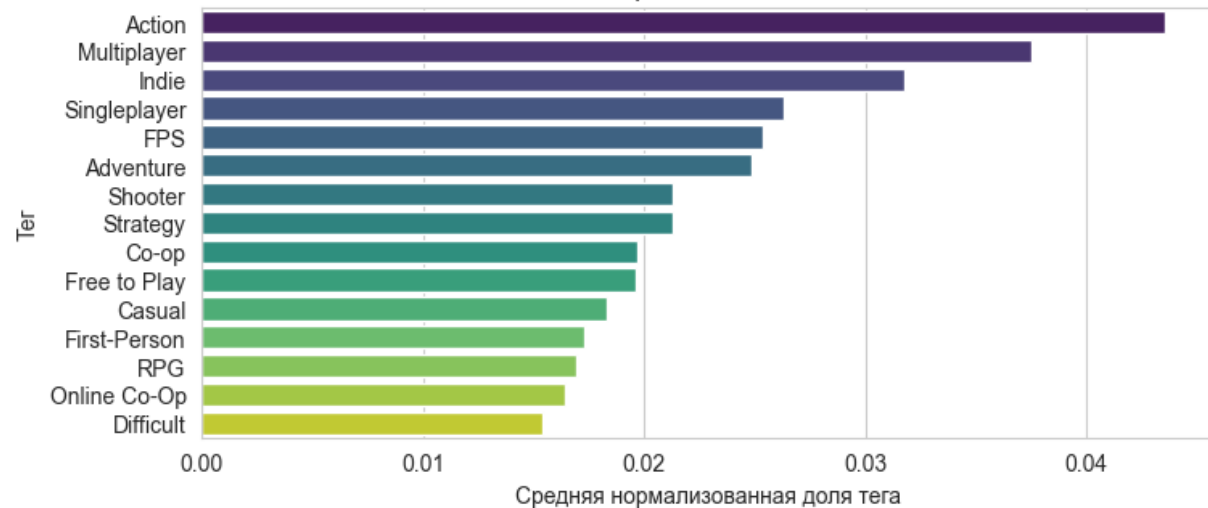
Кластер 4 — Топ-15 тегов



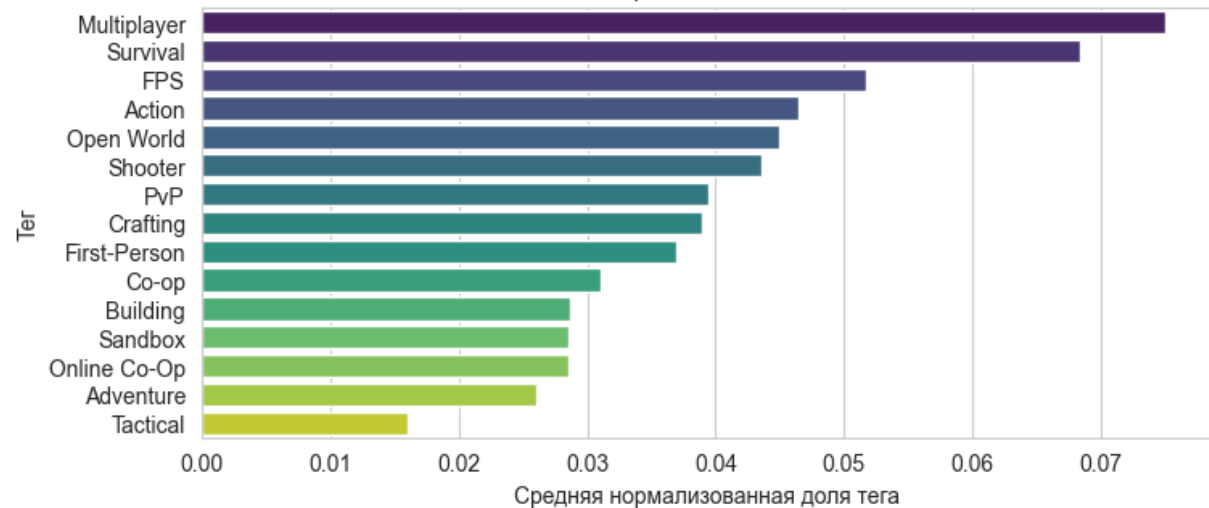
Кластер 5 — Топ-15 тегов



Кластер 6 — Топ-15 тегов



Кластер 7 — Топ-15 тегов



Content-Based Filtering (на основе тегов)

Принцип работы:

- Для каждой игры берём **нормализованные веса тегов** (из SteamSpy / Steam API)
- Для пользователя строим вектор предпочтений: взвешенная сумма тегов по наигранному времени
- Рекомендуем игры с максимальным **cosine similarity** к вектору пользователя

Особенности:

- Использовано **200 самых популярных тегов**
- Учитывается только то, во что пользователь реально играл (по времени)
- Исключаются уже имеющиеся игры

Результаты оценки (Hold-out):

- Precision@10: **0.0173**
- Recall@10: **0.0022**
- Оценка на 1301 пользователе (топ-10% по количеству игр)

Collaborative Filtering (User-Based)

Принцип работы:

- Строится **user-item матрица** (20390 пользователей × 33233 игры)
- Значения = $\lceil 1 + \log(\text{playtimeHours}) \rceil$
- Поиск **похожих пользователей** через KNN (cosine similarity, 15 соседей)
- Рекомендуются игры, которые нравятся похожим пользователям

Особенности:

- Учитывает **поведение** других игроков
- Хорошо работает при наличии достаточной истории

Результаты оценки (Hold-out):

- Precision@10: **0.0689**
- Recall@10: **0.0097**
- Оценка на 1400 пользователях (топ-10% по количеству игр)

Гибридная модель (Content + Collaborative)

Принцип работы:

- Комбинирует два подхода:
 - **Content-Based** (вес 0.3) — теги и предпочтения пользователя
 - **Collaborative** (вес 0.7) — поведение похожих пользователей
- Нормализация оценок + взвешенная сумма

Результаты оценки:

- Precision@10: **0.0676**
- Recall@10: **0.0095**

Преимущества гибридного подхода:

- Лучший баланс между персонализацией и точностью
- Частично решает проблему холодного старта
- Более стабильные и разнообразные рекомендации

Граф друзей - кластеризация и визуализация

Система рекомендаций друзей

Сбор и визуализация графа друзей пользователей Steam

Узлов (пользователей): 20390

Рёбер (связей дружбы): 30358

Средняя степень узла: 2.98

Плотность графа: 0.000146

Изолированных узлов: 2999

Компонент связности: 3258

Размер наибольшей компоненты: 16685

Топ-5 компонент по размеру: [16685, 13, 12, 11, 11]

После фильтрации (только наибольшая компонента):

Узлов: 16685

Рёбер: 29777

Средняя степень узла: 3.57

Плотность графа: 0.000214

Граф очень разреженный: плотность 0.000146 — из всех возможных пар связаны менее 0.015%.

Большинство пользователей имеют 2–3 друга в датасете.

Для анализа берём наибольшую компоненту связности (16 685 узлов) — остальные 3 257 компонент отбрасываем.

Разделение на сообщества - Алгоритм Лоувина

Алгоритм Лоувина - жадный алгоритм кластеризации графа, который итеративно перемещает узлы между сообществами, максимизируя модулярность

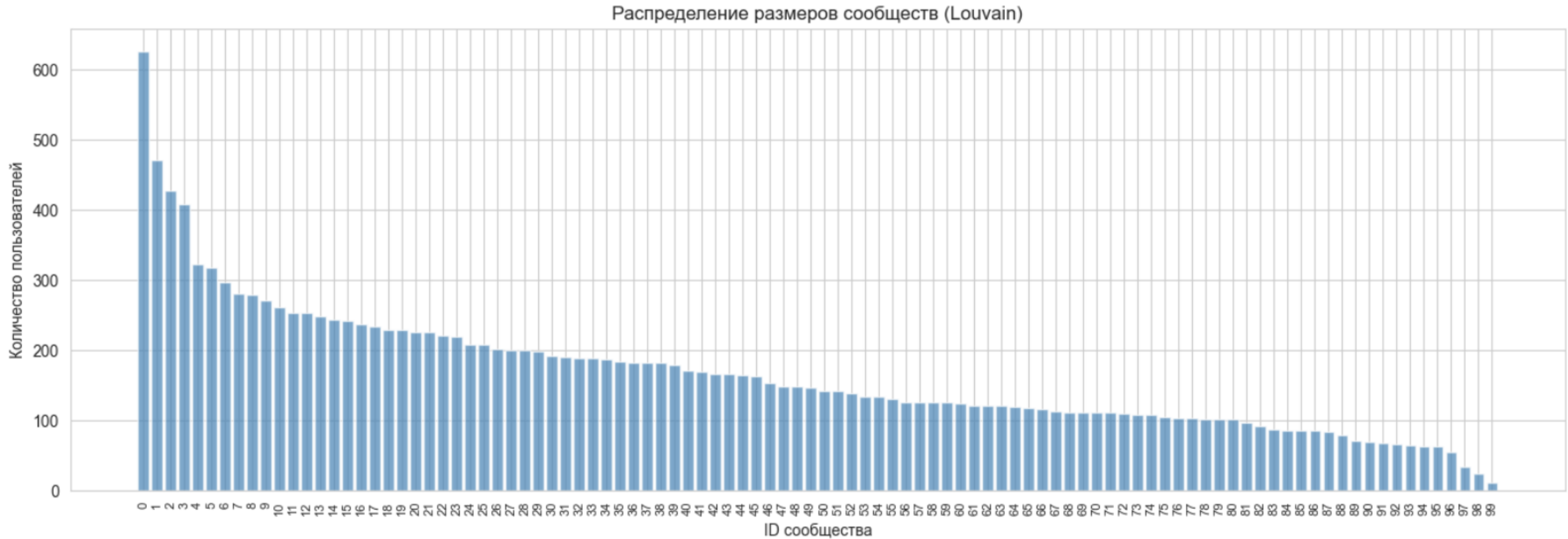
Модулярность(Q) - метрика от -0.5 до 1.0 показывающая насколько хорошо граф разбит на сообщества:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i \cdot k_j}{2m} \right] \delta(c_i, c_j)$$

Алгоритм Лоувина:

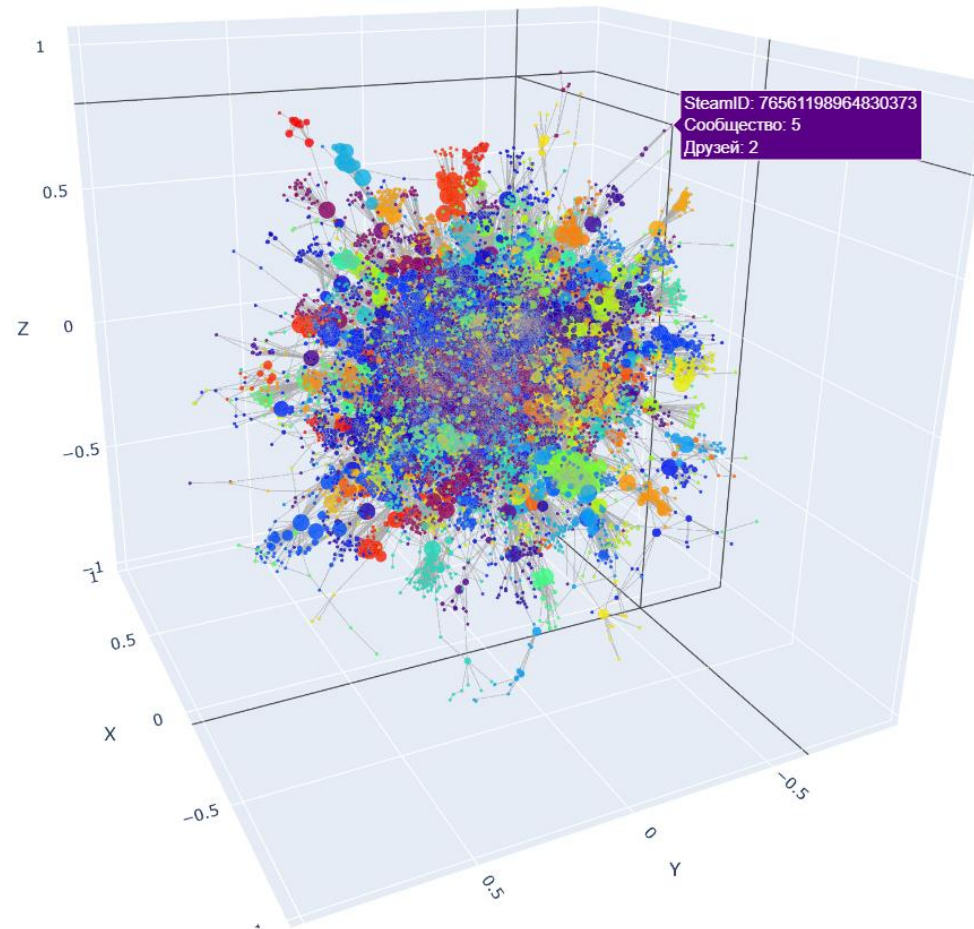
1. Каждый пользователь — сам себе сообщество.
2. Перебираем каждого пользователя: смотрим на его друзей. Пробуем поместить его в сообщество каждого друга. Считаем, вырастет ли модулярность. Перемещаем туда, где модулярность выше.
3. Повторяем пункт 2 по кругу, пока никто не перестанет перемещаться.
4. Сжимаем: каждое найденное сообщество превращаем в один узел. Связи между сообществами становятся рёбрами между этими узлами.
5. Повторяем пункты 2–4 на сжатом графе, пока улучшения не прекратятся.

Разделение на сообщества - Алгоритм Лоувина



Визуализация с разделением на сообщества

Граф друзей с разделением на сообщества



Для каждой пары пользователей вычисляем 10 признаков:

Графовые (7):

Common Neighbors - количество общих друзей

Jaccard Coefficient - доля общих друзей от всех

Adamic-Adar Index - взвешенные общие друзья (редкие ценнее)

Preferential Attachment - произведение степеней

Degree1, Degree2, DegreeDiff - количество друзей, разница в количестве друзей

Игровые (3):

Common Games Count - общие игры

Jaccard Games - доля общих игр

Game Count Diff - разница в размере библиотек

Генерация обучающей выборки

	commonNeighbors	jaccardCoeff	adamicAdar	prefAttachment	sameCommunity	degree1	degree2	degreeDiff	commonGamesCount	jaccardGames	gameCountDiff	label	node1	node2
0	1	0.022727	0.455120	296	1	37	8	29	10	0.061350	103	1	76561198329986045	76561198983870134
1	0	0.000000	0.000000	14	1	14	1	13	4	0.050000	56	1	76561198234818572	76561198106454365
2	6	0.461538	2.294254	90	1	9	10	1	39	0.220339	48	1	76561198211996615	76561198315439262
3	4	0.026846	1.323259	740	1	148	5	143	18	0.151261	43	1	76561198312994520	76561198359563776
4	1	0.008772	1.442695	336	1	112	3	109	56	0.160920	22	1	76561198091117597	76561198149889263
5	1	0.090909	0.333808	27	1	3	9	6	5	0.020921	230	1	76561198053149406	76561198933453135
6	0	0.000000	0.000000	174	0	87	2	85	4	0.023810	150	1	76561198354495262	76561198219969807
7	1	0.142857	0.310667	16	1	4	4	0	11	0.139241	14	1	76561198334350562	76561198800229754
8	0	0.000000	0.000000	324	0	108	3	105	5	0.047170	51	1	76561198125842053	76561198838479645
9	0	0.000000	0.000000	5	1	1	5	4	7	0.145833	19	1	76561198033472783	76561198042826786
10	1	0.024390	1.442695	80	1	40	2	38	1	0.025641	8	1	76561198827482706	76561199362617360
11	0	0.000000	0.000000	36	0	3	12	9	7	0.046358	120	1	76561198427576537	76561198427444809
12	5	0.384615	2.370151	80	1	10	8	2	12	0.122449	12	1	76561198339579741	76561198823426632
13	3	0.025862	2.065881	570	1	114	5	109	52	0.103175	320	1	76561198100621724	76561199062596202
14	2	0.142857	1.116221	63	1	7	9	2	23	0.128492	64	1	76561198080759358	76561198332508229
15	0	0.000000	0.000000	180	0	45	4	41	26	0.047882	485	1	76561198209918107	76561198139700453
16	0	0.000000	0.000000	75	1	25	3	22	6	0.081081	12	1	76561198134225869	76561199085211134
17	0	0.000000	0.000000	113	1	113	1	112	3	0.150000	9	1	76561198342105133	76561199088173377
18	1	0.200000	0.910239	8	1	2	4	2	12	0.093023	89	1	76561198127950116	76561198300130195
19	3	0.272727	1.344477	40	1	10	4	6	1	0.250000	1	1	76561198798806070	76561198818101368

Результаты обучения моделей

=== Random Forest ===
Accuracy: 0.9732
F1-Score: 0.8493
ROC-AUC: 0.9638

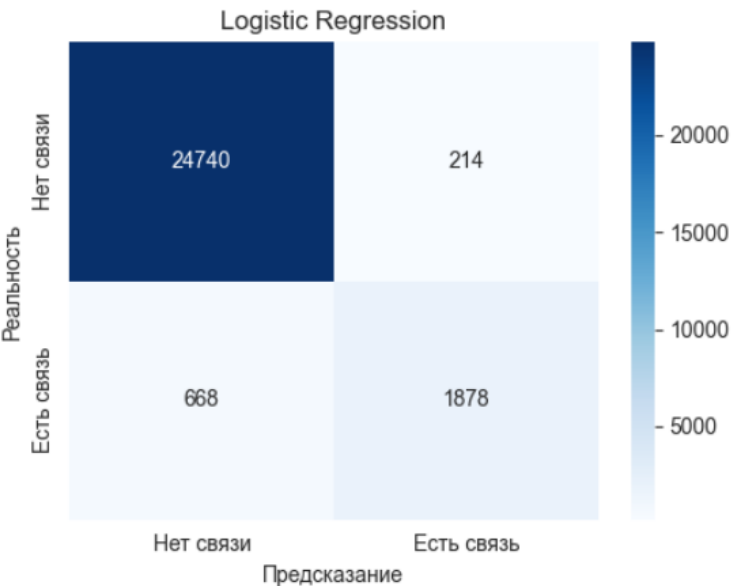
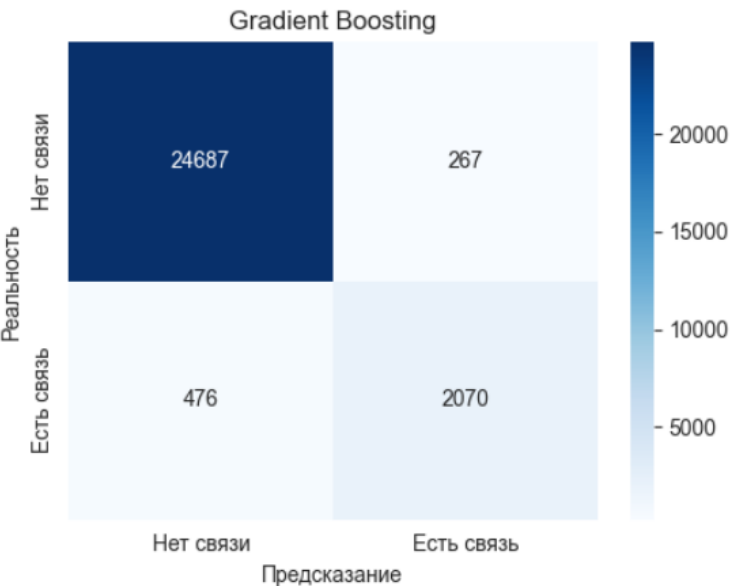
=== Gradient Boosting ===
Accuracy: 0.9730
F1-Score: 0.8478
ROC-AUC: 0.9652

=== Logistic Regression ===
Accuracy: 0.9679
F1-Score: 0.8098
ROC-AUC: 0.9560

	precision	recall	f1-score	support
Нет связи	0.98	0.99	0.99	24954
Есть связь	0.88	0.82	0.85	2546
accuracy			0.97	27500
macro avg	0.93	0.90	0.92	27500
weighted avg	0.97	0.97	0.97	27500

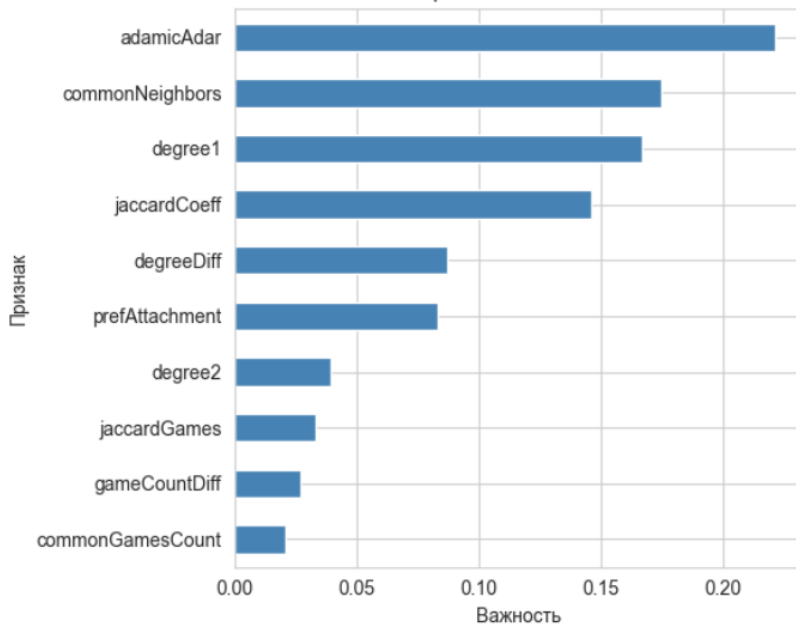
	precision	recall	f1-score	support
Нет связи	0.98	0.99	0.99	24954
Есть связь	0.89	0.81	0.85	2546
accuracy			0.97	27500
macro avg	0.93	0.90	0.92	27500
weighted avg	0.97	0.97	0.97	27500

	precision	recall	f1-score	support
Нет связи	0.97	0.99	0.98	24954
Есть связь	0.90	0.74	0.81	2546
accuracy			0.97	27500
macro avg	0.94	0.86	0.90	27500
weighted avg	0.97	0.97	0.97	27500

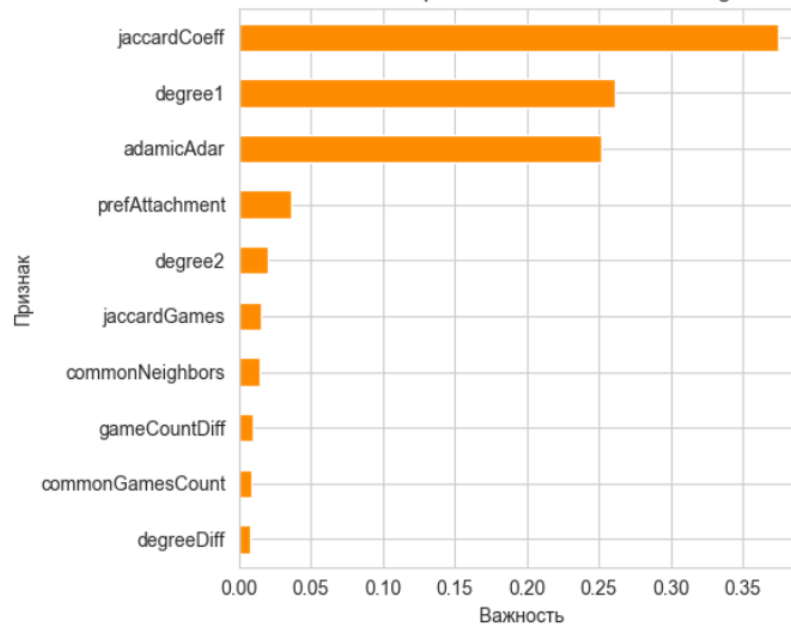


Результаты обученных моделей

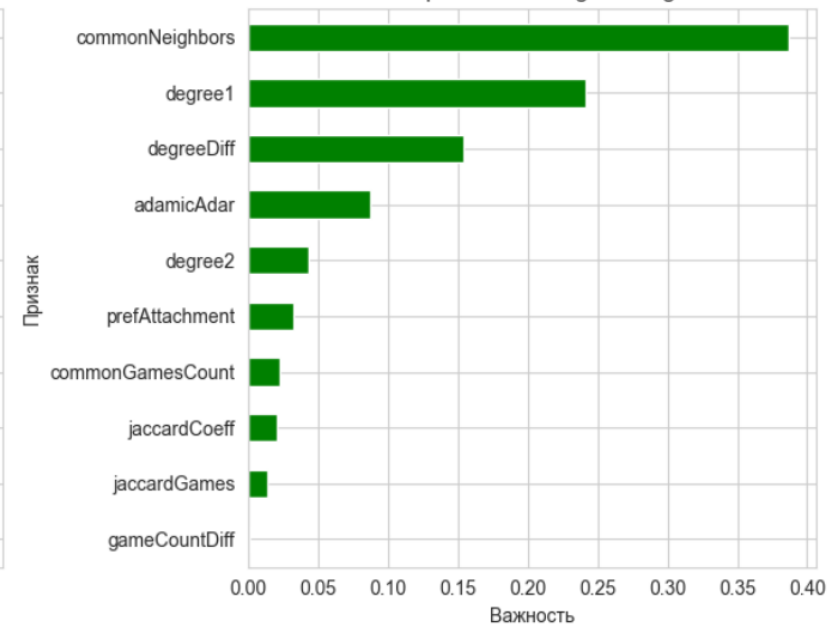
Feature Importance — Random Forest



Feature Importance — Gradient Boosting



Feature Importance — Logistic Regression



Рекомендация друзей

Пользователь: 76561198344519733

Текущих друзей: 10

Сообщество: 88

Топ-10 рекомендованных друзей (Random Forest):

	candidateId	probability	commonNeighbors	commonGames	sameCommunity	jaccardGames
1	76561199236544413	0.999966	3	2	1	0.090909
2	76561199078590793	0.999966	2	7	1	0.120690
3	76561199079003565	0.999948	2	3	1	0.120000
4	76561198365351032	0.996216	2	7	1	0.095890
5	76561198387468435	0.992827	1	6	0	0.098361
6	76561198255619051	0.990608	1	8	0	0.190476
7	76561198971265954	0.985075	1	3	1	0.066667
8	76561198797088177	0.985075	1	4	0	0.075472
9	76561199489640612	0.983018	1	8	1	0.080000
10	76561199135955991	0.980448	1	2	0	0.068966

Топ-10 рекомендованных друзей (Gradient Boosting):

	candidateId	probability	commonNeighbors	commonGames	sameCommunity	jaccardGames
1	76561199236544413	0.998670	3	2	1	0.090909
2	76561199078590793	0.997916	2	7	1	0.120690
3	76561198965870045	0.997899	1	8	0	0.070796
4	76561198255619051	0.997733	1	8	0	0.190476
5	76561199079003565	0.997509	2	3	1	0.120000
6	76561198365351032	0.997276	2	7	1	0.095890
7	76561199135955991	0.996748	1	2	0	0.068966
8	76561198387468435	0.996289	1	6	0	0.098361
9	76561199489640612	0.978922	1	8	1	0.080000
10	76561198267141596	0.975704	1	6	1	0.122449

Топ-10 рекомендованных друзей (Logistic Regression):

	candidateId	probability	commonNeighbors	commonGames	sameCommunity	jaccardGames
1	76561199236544413	0.999999	3	2	1	0.090909
2	76561199078590793	0.999791	2	7	1	0.120690
3	76561199079003565	0.999648	2	3	1	0.120000
4	76561198365351032	0.999638	2	7	1	0.095890
5	76561198965870045	0.987765	1	8	0	0.070796
6	76561199135955991	0.984016	1	2	0	0.068966
7	76561198255619051	0.969716	1	8	0	0.190476
8	76561198387468435	0.948069	1	6	0	0.098361
9	76561198808335092	0.892574	1	6	1	0.081081
10	76561198267141596	0.889450	1	6	1	0.122449



| 300
ЛЕТ СПбГУ



Спасибо за внимание

Ссылка на репозиторий проекта:

<https://github.com/FasterXaos/Steam-ML>

